Linux Systems and Performance

Sungyong Park, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr

E-mail: parksy@sogang.ac.kr



Course Introduction: Linux Systems and Performance

Linux Internals, Observability, and Performance Analysis

Course Objectives

Understand what happens underneath a webOS application (Linux) — and learn how to observe it.

14-WEEK COURSE ROADMAP

PART 1 · Week 1–3 (3 weeks)
Linux Execution Model
Prof. Sungyong Park

PART 2 · Week 4–6 (3 weeks)
CPU Performance & Scheduling
Prof. Youngjae Kim

PART 3 · Week 7–10 (4 weeks)
Memory Management
Prof. Sungyong Park

PART 4 · Week 11–14 (4 weeks)
File System & Disk I/O
Prof. Youngjae Kim

HOW WE WILL LEARN

LEARN & BUILD → **OBSERVE** → **DIAGNOSE** → **EXPLAIN**

WEEKLY FORMAT
3 H Lecture + 1 H Hands-on lab

PRACTICE ENVIRONMENT
General Linux

EVALUATION
Final Exam, Project (1 or 2), Homework (after each lab)

Course Introduction: Instructors

Instructors

Linux Systems and Performance (for LG webOS Developers)

Systems for a Better World



Sungyong Park, Ph.D.

Professor
Department of Computer Science and Engineering
Sogang University

Distributed Systems
Scalable and reliable systems

Cloud & Edge Computing
Cloud infrastructure and edge systems

AI Systems
Platforms and infrastructure for AI

parksy@sogang.ac.kr | 02-705-8929

Lecture Topics (This Course)
Linux Execution Model · Memory Management



Youngjae Kim, Ph.D.

Professor
Department of Computer Science and Engineering
Sogang University

Operating Systems
OS internals and system software

File & Storage Systems
File systems, storage, and I/O systems

AI Systems
Platforms and infrastructure for AI

youkim@sogang.ac.kr | 02-705-8933

Lecture Topics (This Course)
CPU & Scheduling · File System & Disk I/O

SOGANG UNIVERSITY
Department of Computer Science and Engineering

DISCOS Data Intensive AI Computing and Systems Laboratory
https://discos.sogang.ac.kr

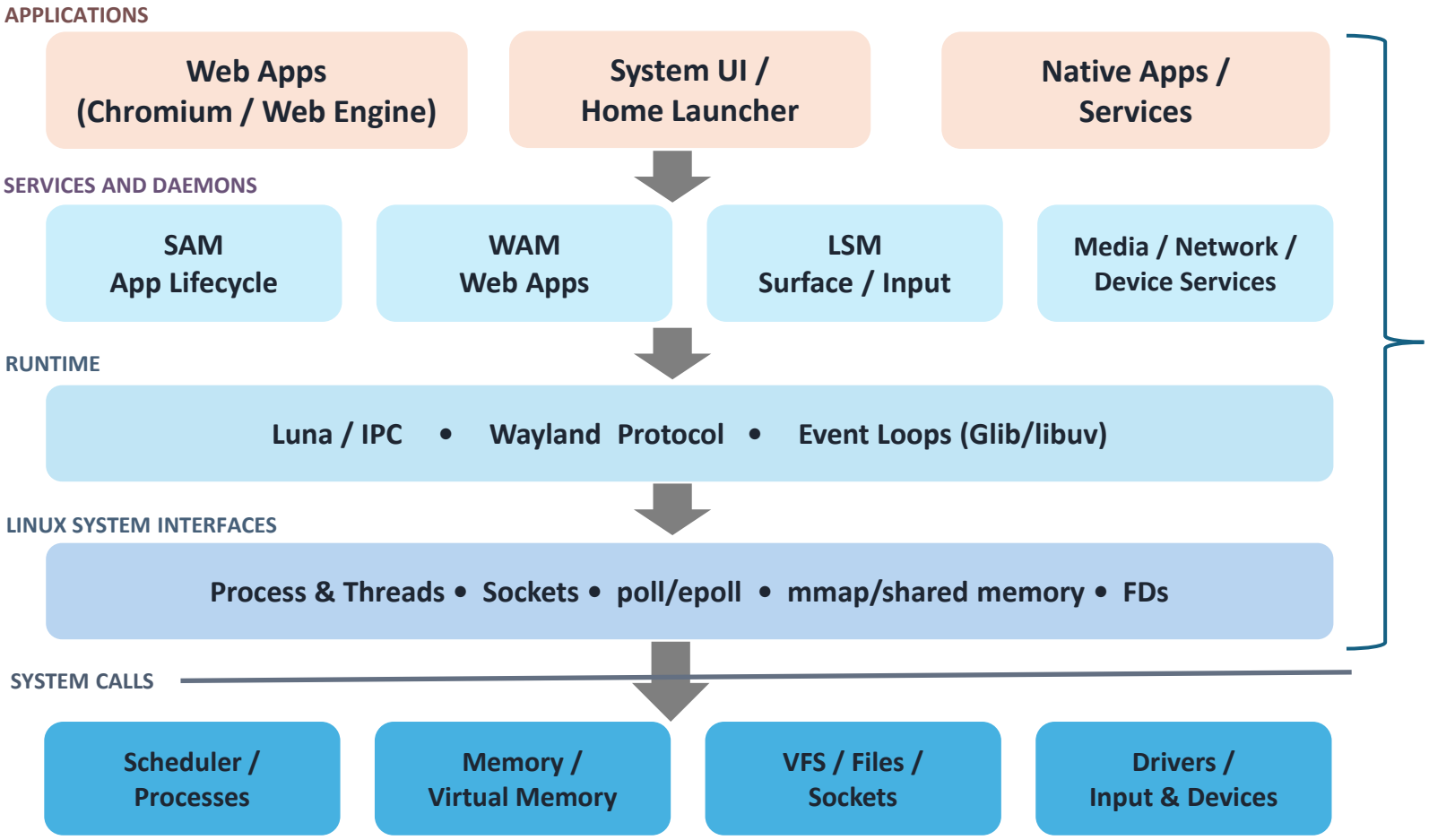
Systems · Data · AI
for a Better World



LG전자 | DISCOS Data-Intensive AI Computing & Systems Laboratory

Overview of webOS Platform on Linux

The webOS platform consists of **various applications and system services** running as **processes / threads** on top of the **Linux kernel**. These processes rely on **Linux system interfaces** for execution, communication, event handling, and resource access.



Linux Mechanisms Used In webOS

- **Processes & Threads**
  Apps, services, daemons
- **System Calls**
  Access to kernel services
- **Exceptional Events**
  Interrupts, signals, faults
- **IPC & FDs**
  Sockets, shared memory, files
- **Event Handling**
  Blocking, readiness, event loops

Need to Know How these actually execute on Linux.

From Application View to Kernel View

A 14-week journey: first understand **how programs run**, then **how Linux manages the resources** underneath.

APPLICATION PERSPECTIVE

PART 1
Linux Execution Model
Weeks 1–3

Q: How do Linux applications run?

SYSTEM CALL / KERNEL BOUNDARY

KERNEL PERSPECTIVE

PART 2
CPU Management
Weeks 4–6

Q: How does Linux schedule and share CPU time?

PART 3
Memory Management
Weeks 7–10

Q: How does Linux allocate, cache and reclaim memory?

PART 4
File System & Storage
Weeks 11–14

Q: How does Linux manage files, caching and disk I/O?

Weeks 1–3: How does my program run? → **Weeks 4–14: How does the kernel make it run?**

Linux Systems and Performance

PART 1: Linux Execution Model

Week 1: Process Execution and System Call

Sungyong Park, Ph.D.

Data-Intensive AI Computing and Systems (DISCOS) Laboratory

Department of Computer Science and Engineering, Sogang University

URL: https://discos.sogang.ac.kr

E-mail: parksy@sogang.ac.kr



LG전자 | DISCOS
Data-Intensive AI Computing &
Systems Laboratory

PART 1 : Three Views of Linux Execution

From **process creation** → **exceptional events** → **communication & event-driven execution**

LECTURE

WEEK 1	WEEK 2	WEEK 3
Process ↔ Kernel	**Event → Process**	**Process ↔ Process**
How are processes created and how do they interact with Linux?	What happens when normal control flow is disrupted?	How do processes communicate and handle multiple events?
Process / Thread	Exceptional Control Flow	Blocking / Non-blocking
fork • exec • wait	Interrupt / Exception	eventfd • epoll
System Call	Signal	Event Loop
File Descriptor	Handler / Return	IPC / Shared Memory

LAB

Tiny Shell v1 + Observe	**Signal-aware Shell + Observe**	**Event-driven Server + Observe**

LG전자 | DISCOS Data-Intensive AI Computing & Systems Laboratory

PART 1: Linux Execution Model

- **Week 1**

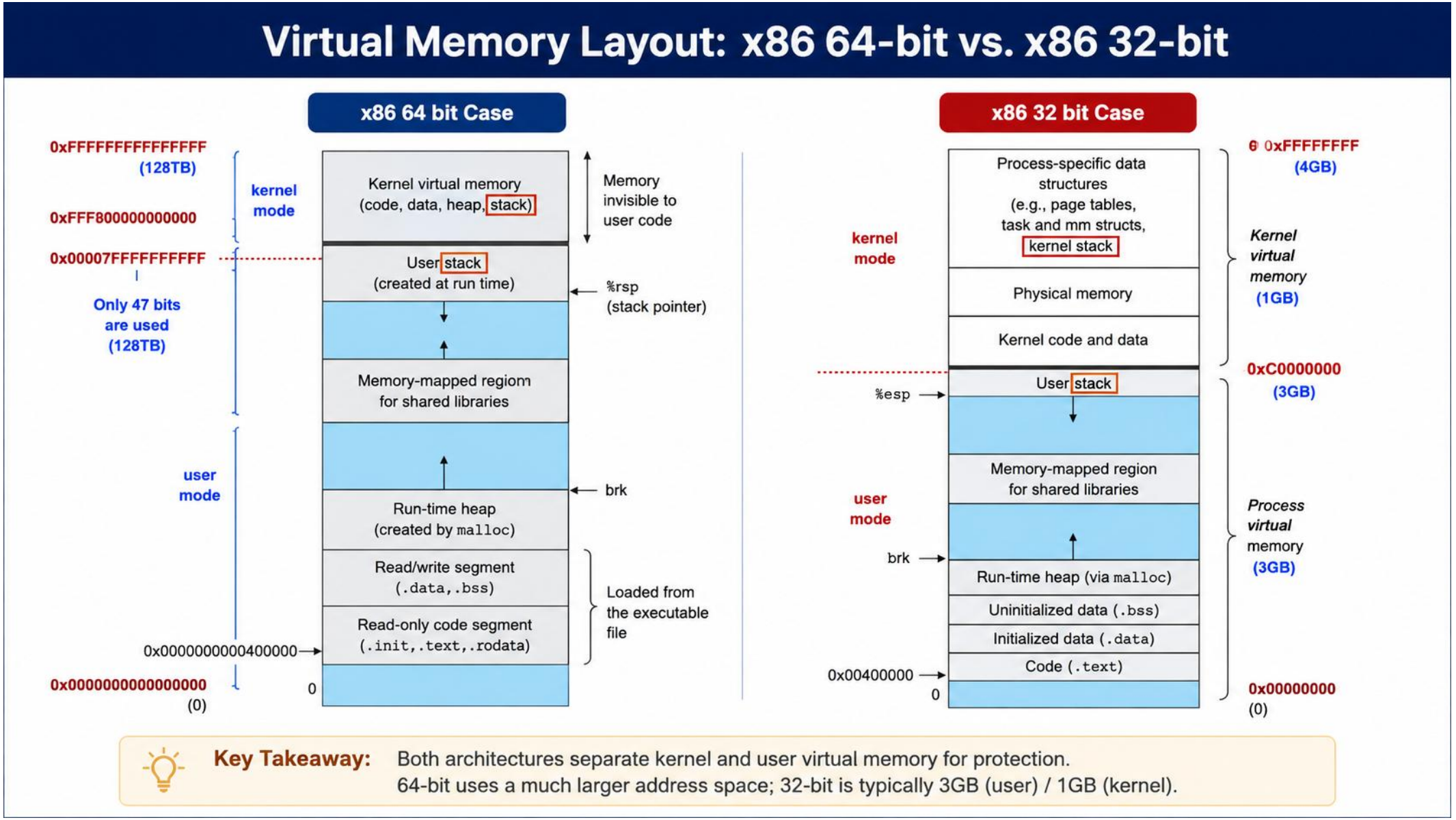
- **Processes and Threads**
  - How Linux represents and executes tasks
- **System Calls**
  - How programs request services from the kernel
- **File Descriptors**
  - How Linux provides a unified interface to I/O resources

Processes

- **A *process* is an instance of a running program**
  - Program in execution
  - Not the same as “program” or “processor”
  - A program is passive, whereas a process is a live execution context

- **Process provides each program with two key abstractions:**
  - ***Virtual Address Space (VAS)***
    - Each process seems to have exclusive use of main memory
    - Provided by kernel mechanism called *virtual memory*
  - ***Logical Control Flow***
    - Each process seems to have exclusive use of the CPU
    - Provided by kernel mechanism called *context switching*

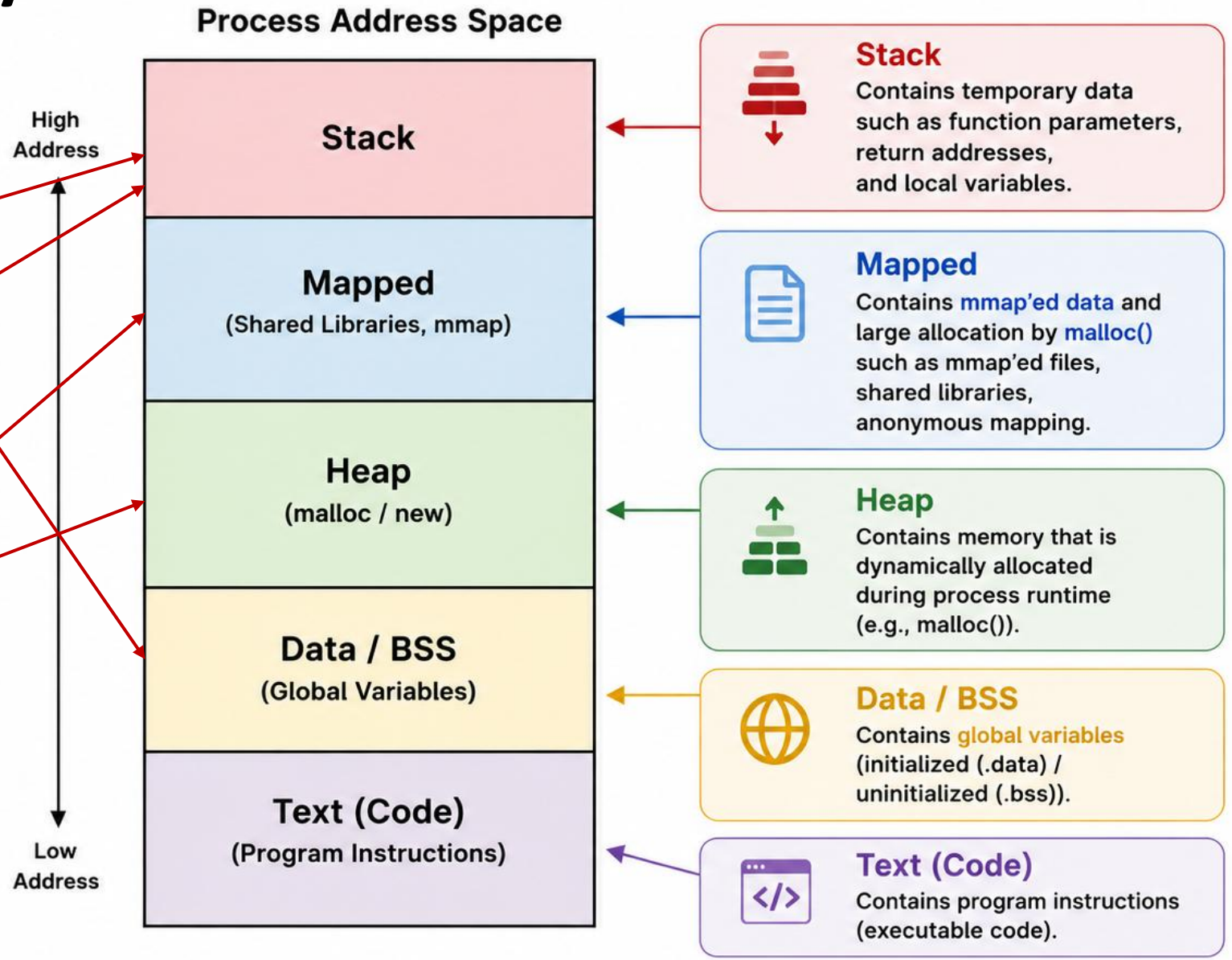
Virtual Address Space (VAS)



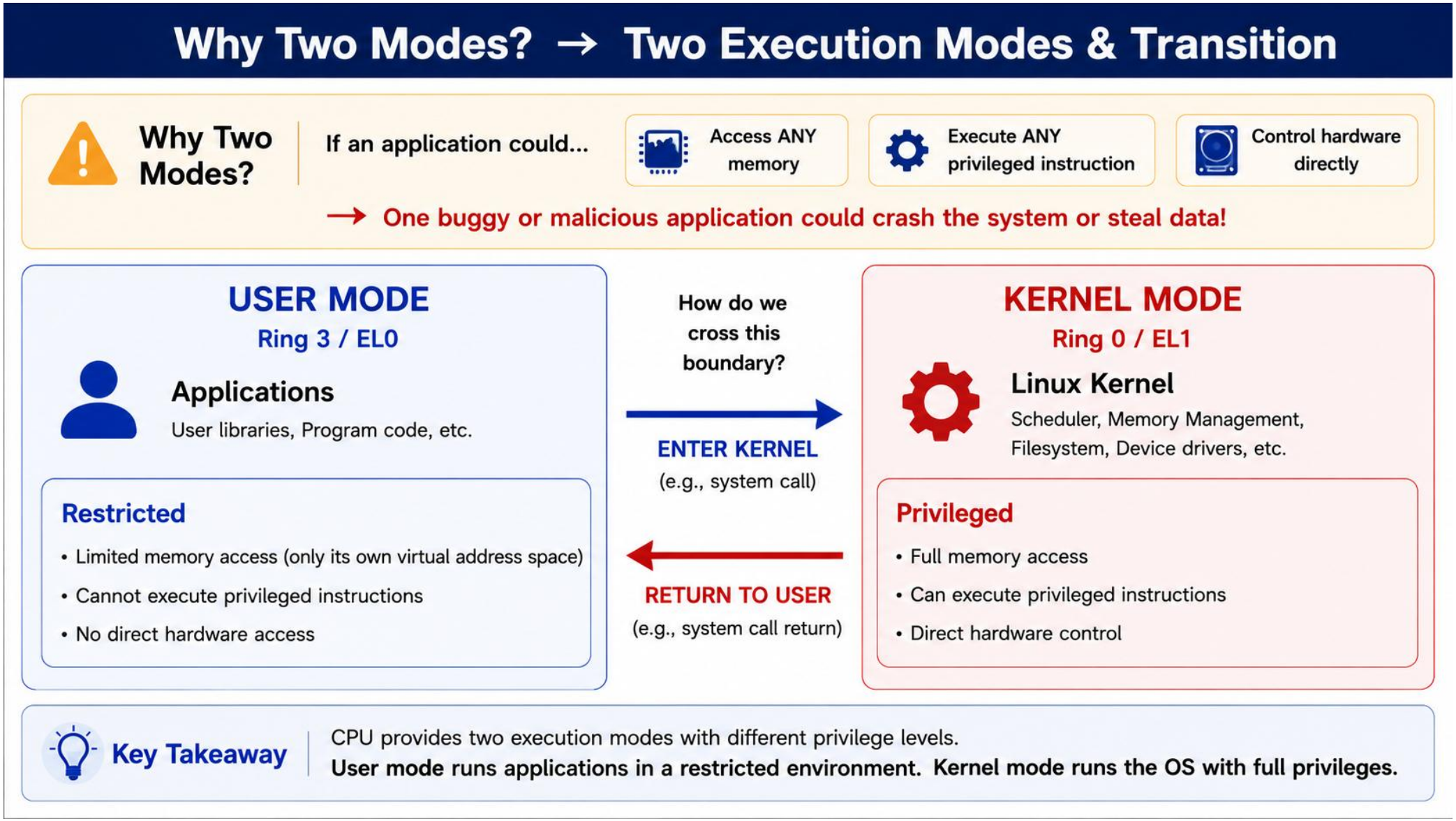
Process in Memory

```c
int tmp;
int cnt = 0;
int main(void)
{
    int i, j, sum;
    char *heapdata, *mapped;

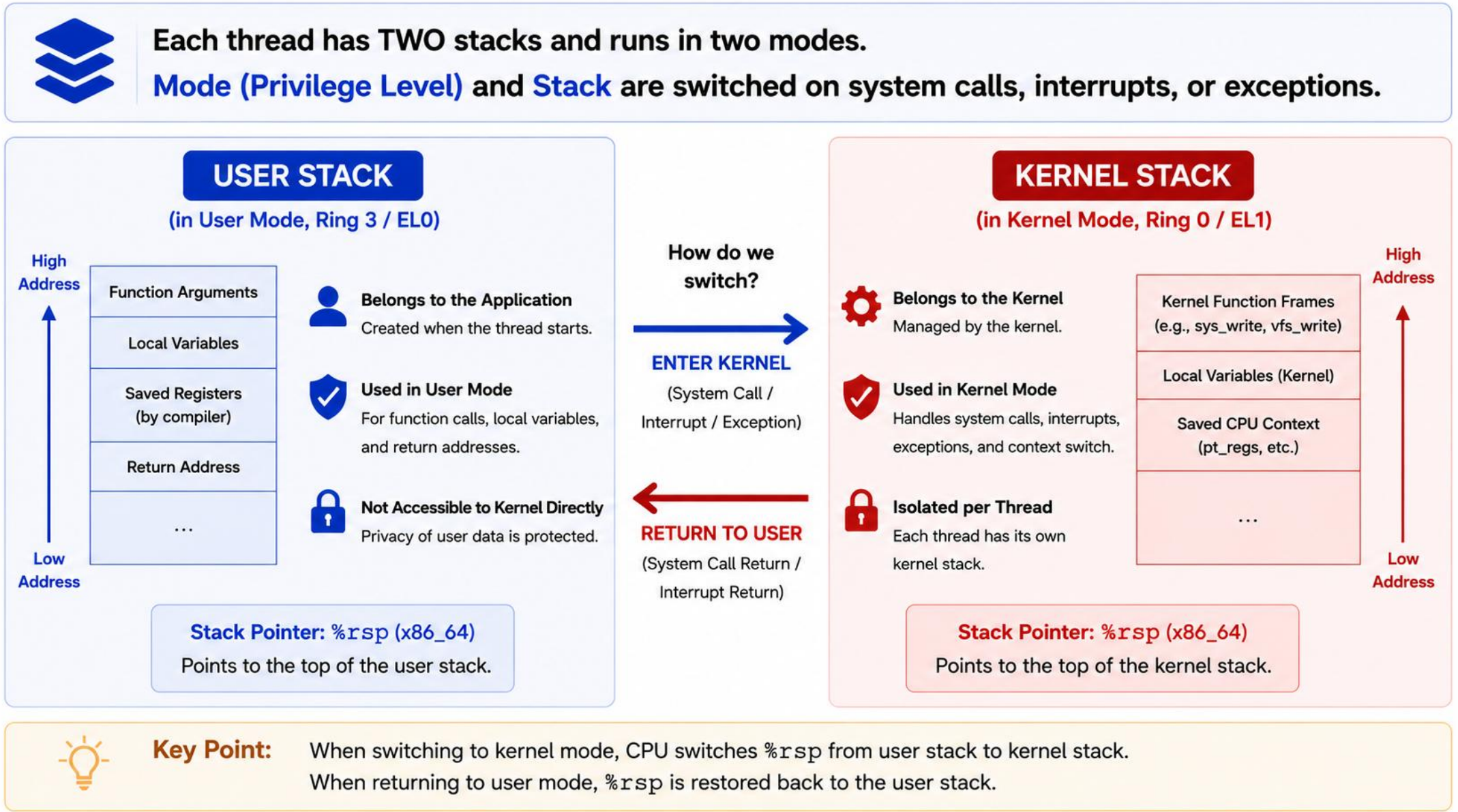
    i = 10;
    j = 20;
    sum = add(i, j, ++cnt);
    printf("Sum is %d\n", sum);
    mapped = mmap(...);
    heapdata = (char *) malloc(sum);
    ....
}
int add(int k, int l, int m)
{
    int sum;
    sum = k + l + m;
    return sum;
}
```



Two Execution Modes

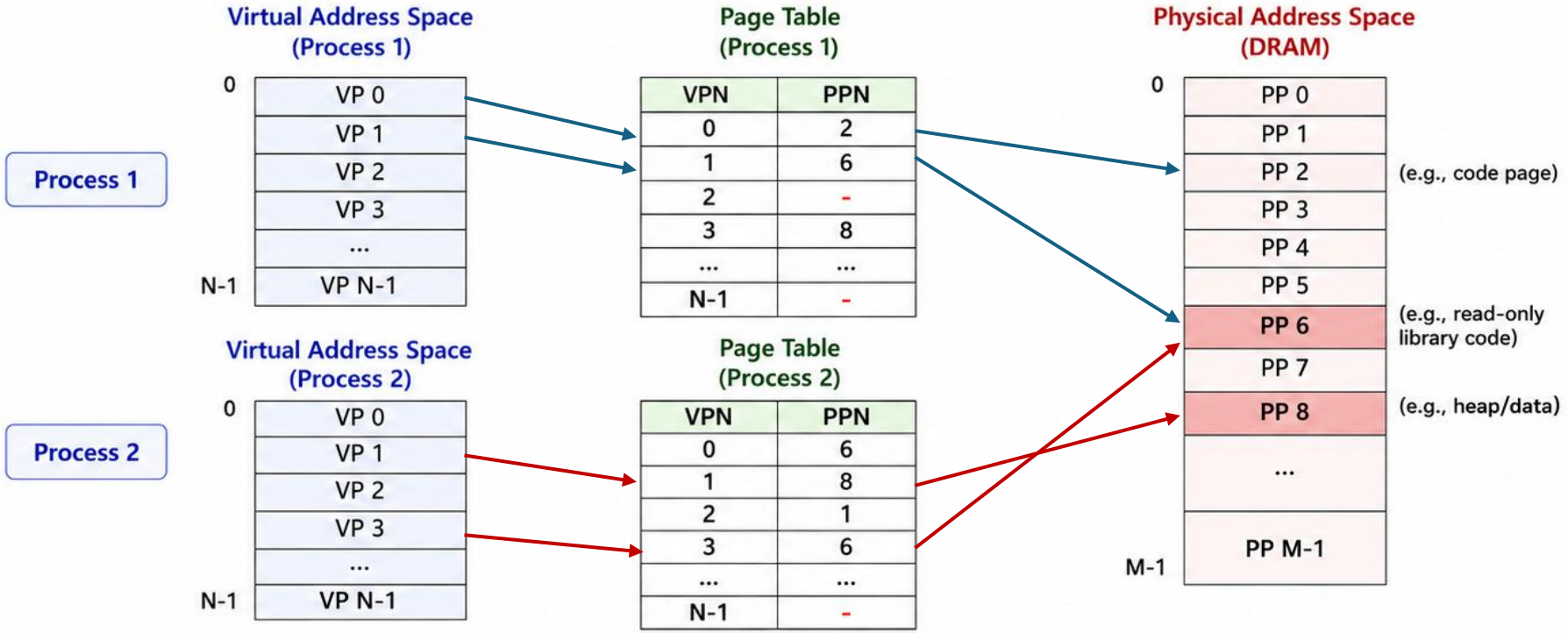


User Stack vs. Kernel Stack



VM as a Tool for Memory Management

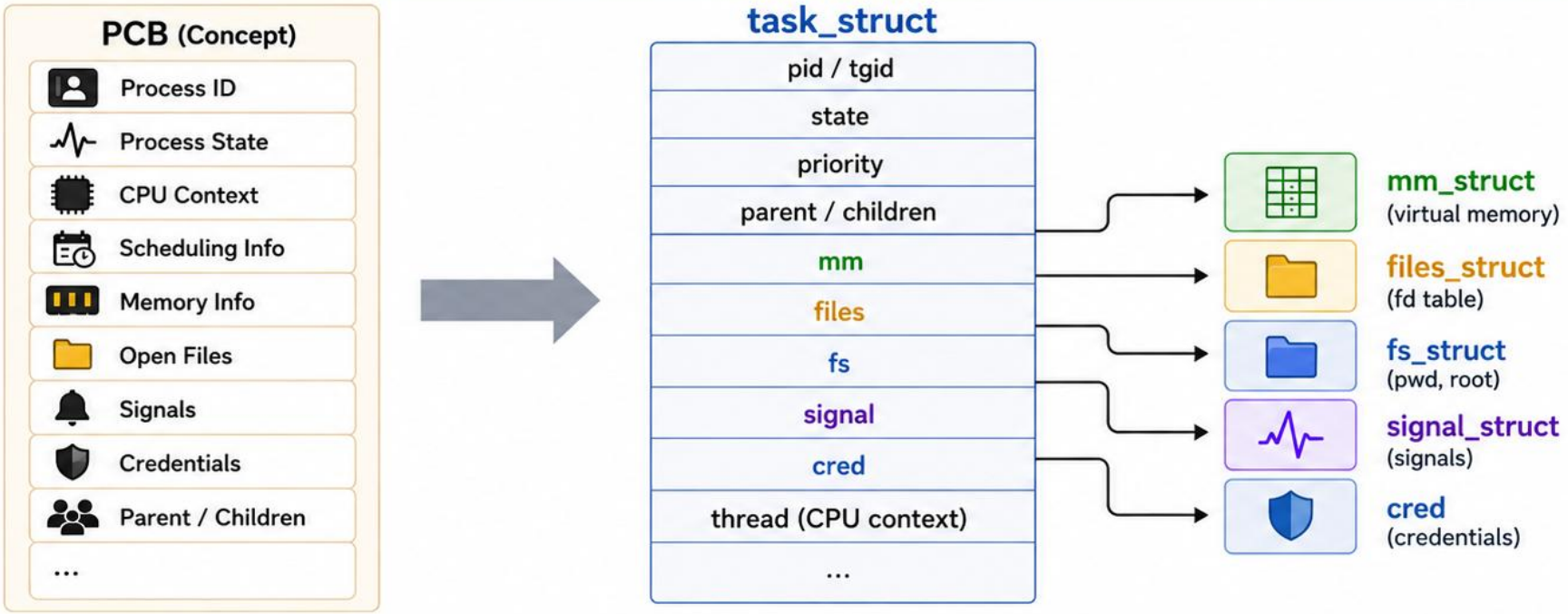
- Virtual/physical address spaces are partitioned into fixed-size pages
- Each virtual page can be mapped to any physical page
- A virtual page can be stored in different physical pages at different times
- Multiple virtual pages can map to the same physical page (e.g., PP 6)



Each process has its own page table.
The page table maps virtual pages (VPN) to physical pages (PPN).
Multiple virtual pages (from same or different processes) can map to the same physical page.

task_struct in Linux

- Linux represents each task using a kernel structure called **struct task_struct**
- Linux implementation of the **Process Control Block (PCB)** concept
- Contains the task's identity, state, scheduling, and relationship information
- Points to kernel structures managing memory, files, signals, and credentials
- **Several KB** (~4–10 KB, depending on the kernel configuration); allocated from a dedicated slab cache

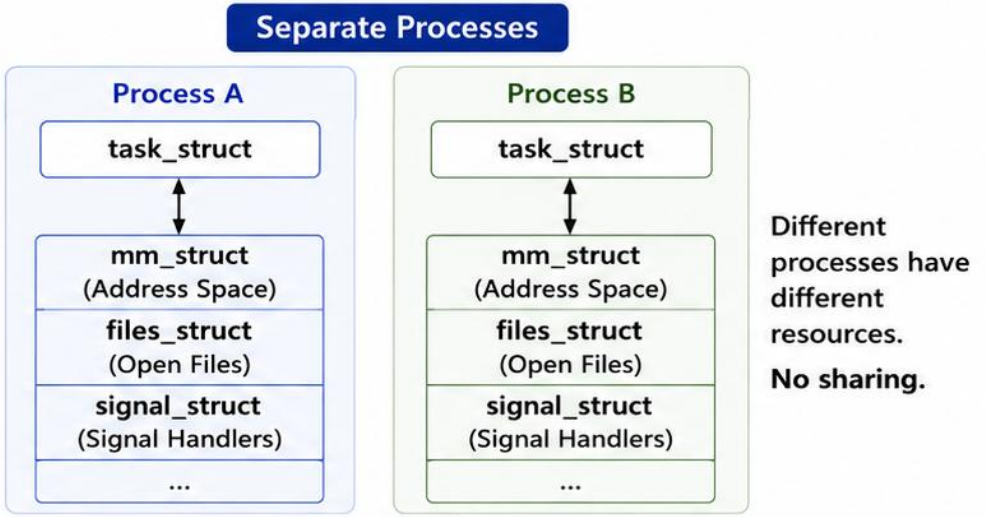


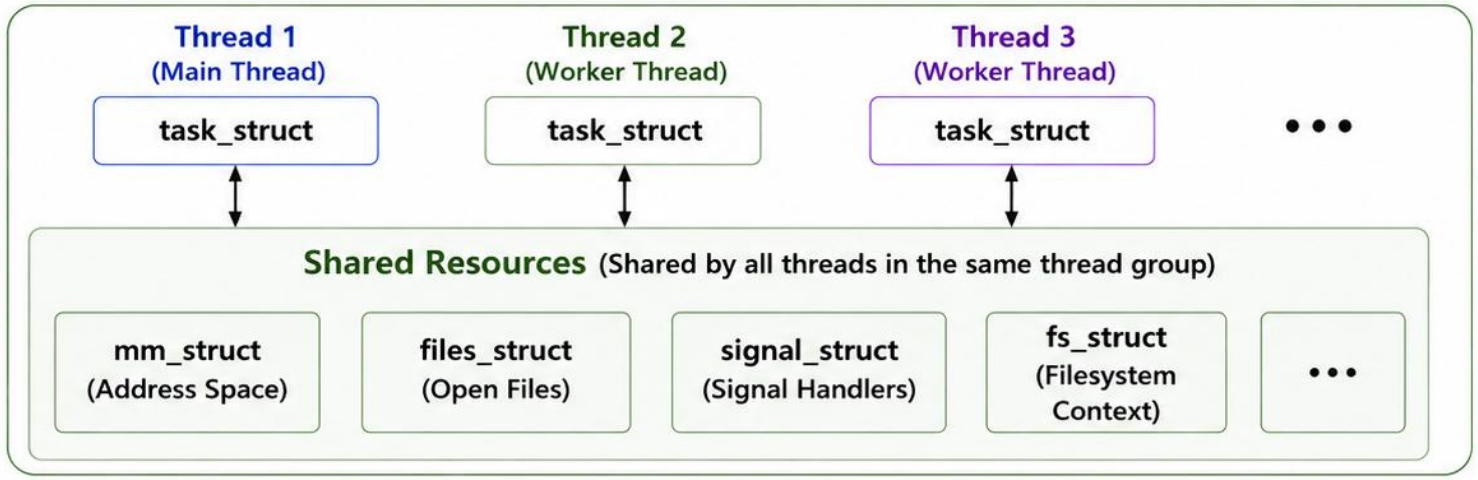
Threads in General

- A control flow within a process
- Multiple threads can be created within a same process and share code, address space, and other process resources
- An individual thread has at least its own execution context such as registers, PC (program counter), SP (stack pointer) and stack
  - Each thread has its own thread id (TID)
- Each thread is individually scheduled by OS similar to process scheduling
- Thread creation and context switching can be cheaper because threads share many process resources, including the address space

What is a Thread in Linux?

- In Linux, there is no special "thread" object in the kernel. The kernel sees everything as tasks.
- A thread is simply a task that shares certain resources with other tasks in the same thread group.
- From the kernel's point of view, a thread and a process are fundamentally the same: just tasks.





Threads (tasks) in the same thread group share many resources, especially the address space.

Each thread still has its own execution context:

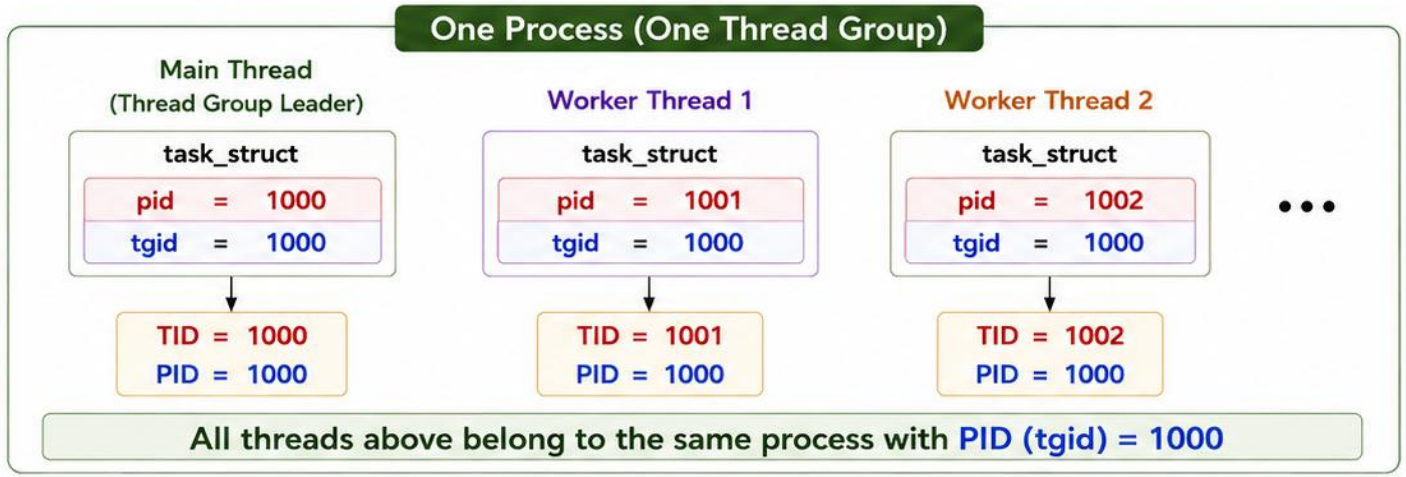
- task_struct
- kernel stack
- CPU registers, PC, SP, etc.
- user stack

➡ A Linux thread is not a special entity. It is just a task that shares specific fields with other tasks.

Task Identification in Linux

- In Linux, every task (process or thread) has a `task_struct` with two important ID fields.
  - `pid` : the task's unique ID in the system → Thread ID (TID)
  - `tgid` : the thread group ID → Process ID (PID)
- Threads in the same thread group share the same `tgid` (i.e., same Process ID).
- The main (leader) thread of a thread group has `pid == tgid`.

```
struct task_struct {
  ...
    pid_t   pid;    // Thread ID (TID)
    pid_t   tgid;   // Thread Group ID (Process ID)
  ...
}
```



- **pid (Thread ID, TID)**
  - Unique ID of a task in the system.
  - Used by the kernel to identify and schedule the task.
- **tgid (Thread Group ID, Process ID, PID)**
  - ID of the thread group (process).
  - All threads in the same process have the same `tgid`.
- **Main (leader) thread**
  - The only thread whose `pid == tgid`.

PID in user space (e.g., kill PID)

How Linux Shows IDs

```
$ ps -elf
UID    PID   PPID   LWP   TTY     CMD
1000   1000  900    1000  pts/0   main
1000   1000  900    1001  pts/0   worker-1
1000   1000  900    1002  pts/0   worker-2
```

PID column (same for all threads)

LWP column (equals TID / pid)

/proc View

```
$ ls -l /proc/1000/task
total 0
dr-x------ 2 user user 0 May 12 10:00 1000
dr-x------ 2 user user 0 May 12 10:00 1001
dr-x------ 2 user user 0 May 12 10:00 1002
```

Entries are TIDs.

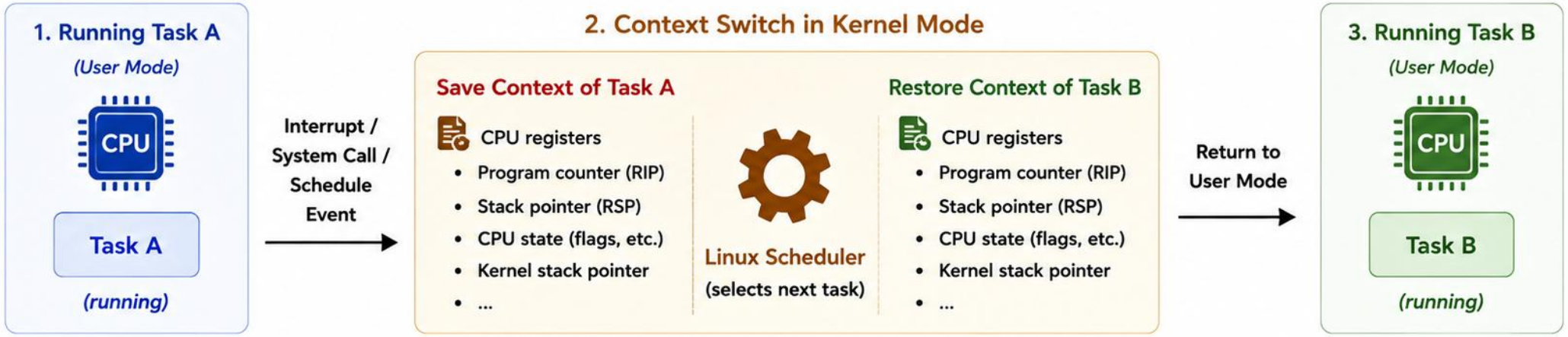
Each thread has its own `/proc/<tgid>/task/<tid>` directory.

Logical Control Flow – Context Switch

- A **context switch** is when the CPU switches from running one task to running another task.
- The kernel **saves** the context of the current task and **restores** the context of the next task.
- Triggered by timer interrupt, blocking I/O, wakeup, or system call.

What Triggers a Context Switch?

- ✓ Timer interrupt (time slice expired)
- ✓ Blocking system call / I/O wait
- ✓ A higher-priority task becomes runnable
- ✓ Task voluntarily yields the CPU



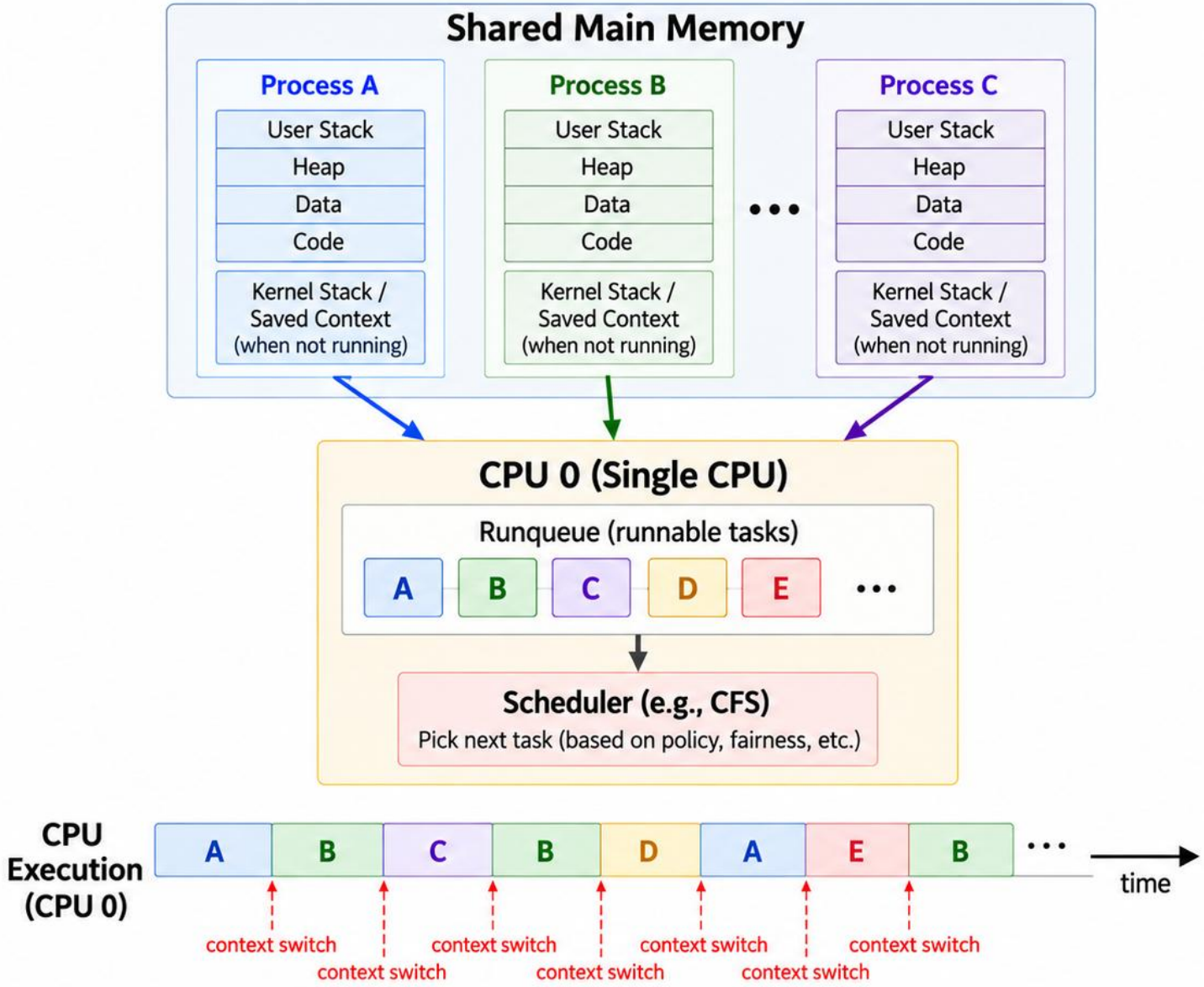
What Gets Saved / Restored?

CPU Registers (RAX, RBX, RCX, ...)	Program Counter (RIP)	Stack Pointer (RSP)	CPU State (flags, etc.)	Kernel Stack Pointer	Other Kernel Data

Key Points

- ✓ Context switch does not happen in user mode.
- ✓ The kernel controls the switch.
- ✓ Costly: saving/restoring state takes time.
- ✓ Keep it low for good performance!

Concurrency: Running on a Single CPU



Key Points

Single CPU + Scheduling

Only one task runs at a time. The scheduler (e.g., CFS) picks the next runnable task from a single runqueue.

Frequent Context Switches

The kernel switches between tasks very frequently (short time slices).

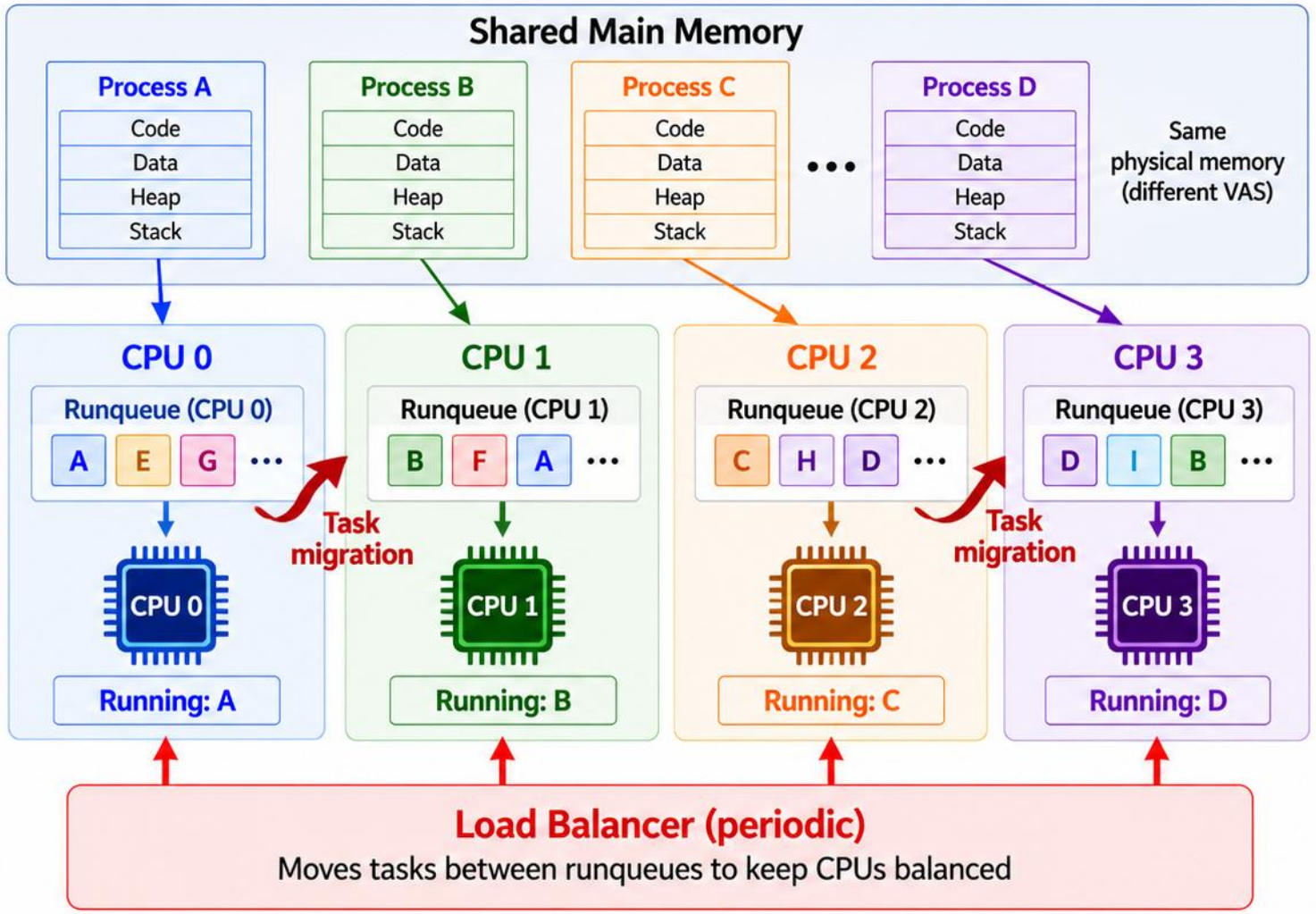
Concurrency

Many tasks make progress over time, giving the illusion of running "simultaneously."

Overlap with I/O

When a task blocks for I/O, other tasks can run, improving CPU utilization.

Parallelism: Running on Multiple CPUs or Cores



Key Points

Per-CPU Runqueue
Each CPU has its own runqueue of runnable tasks.

Load Balancing
Periodically moves tasks between CPUs (task migration) to keep load balanced.

Cache Effect
A migrated task may lose CPU cache (cold cache), which can hurt performance.

True Parallelism
Different CPUs execute different tasks at the same time.

Multiple tasks run in **parallel** on multiple CPUs, with periodic **load balancing** via task migration.

perf: Observe Linux Execution

1 What can perf show?

Scheduling

Which task ran on which CPU when context switches happened.

Performance counters

CPU cycles, instructions, context switches, migrations, page faults, etc.

CPU hotspots

Where CPU time is spent (functions, call stacks) across **user** and **kernel**.

2 Example commands

A Scheduling timeline

Record and show scheduling events

```
$ perf sched record ./app
$ perf sched timehist
```

B Basic performance counters

Run a program and show key metrics

```
$ perf stat ./app
```

C CPU hotspot (with call stack)

Profile and show where CPU time is spent

```
$ perf record -g ./app
$ perf report
```

3 Sample output and what it means

A perf sched timehist (excerpt)

```
time     cpu   task       pid     event
0.001    0     app        1234    sched_in
0.003    0     worker     1235    sched_out
0.004    1     app        1234    sched_in
...
```

- Which task ran on which CPU when context switches happened.

B perf stat (excerpt)

```
Performance counter stats for './app':
   1,234,567,890    cycles
     987,654,321    instructions
          12,345    context-switches
           5,678    page-faults
   1.234567890    seconds time elapsed
```

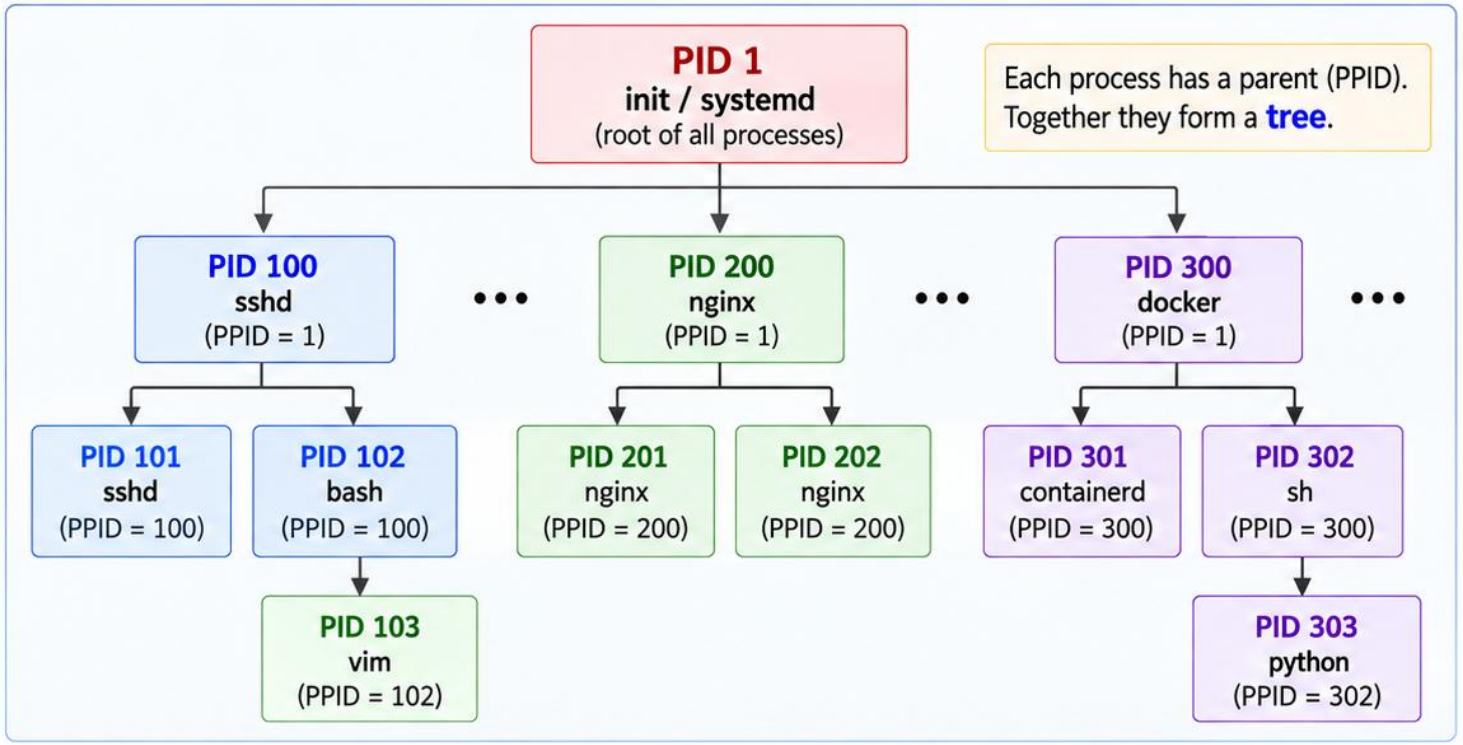
- How much work was done (e.g., cycles, instructions, context switches, page faults).

C perf report (excerpt with call stack)

```
Overhead  Command  Shared Object      Symbol
35.2%     app      app                [.] compute_data()
                                        └ process_data()
                                          └ main()
                                            └ _libc_start_main()
18.7%     app      libc.so.6          [.] qsort()
                                        └ sort_items()
                                          └ process_data()
                                            └ main()
 8.6%     app      [kernel.kallsyms]  [k] vfs_write()
                                        └ __x64_sys_write()
                                          └ do_syscall_64()
                                            └ write()
                                              └ save_result()
...
```

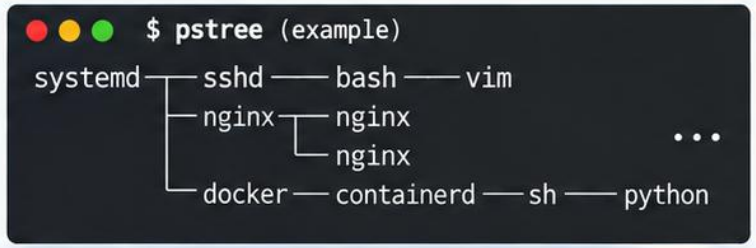
- Hot functions with their call stacks.
- Shows caller → callee path.
- Includes user and kernel functions.
- Percentages are relative to total samples.

Linux Process Hierarchy



View Process Hierarchy

```
$ pstree
```



Key Points

Parent–Child Relationship

Each process has a parent (PPID). Processes form a **tree hierarchy**.

Root Process

PID 1 (init / systemd) is the root of all processes.

View with pstree

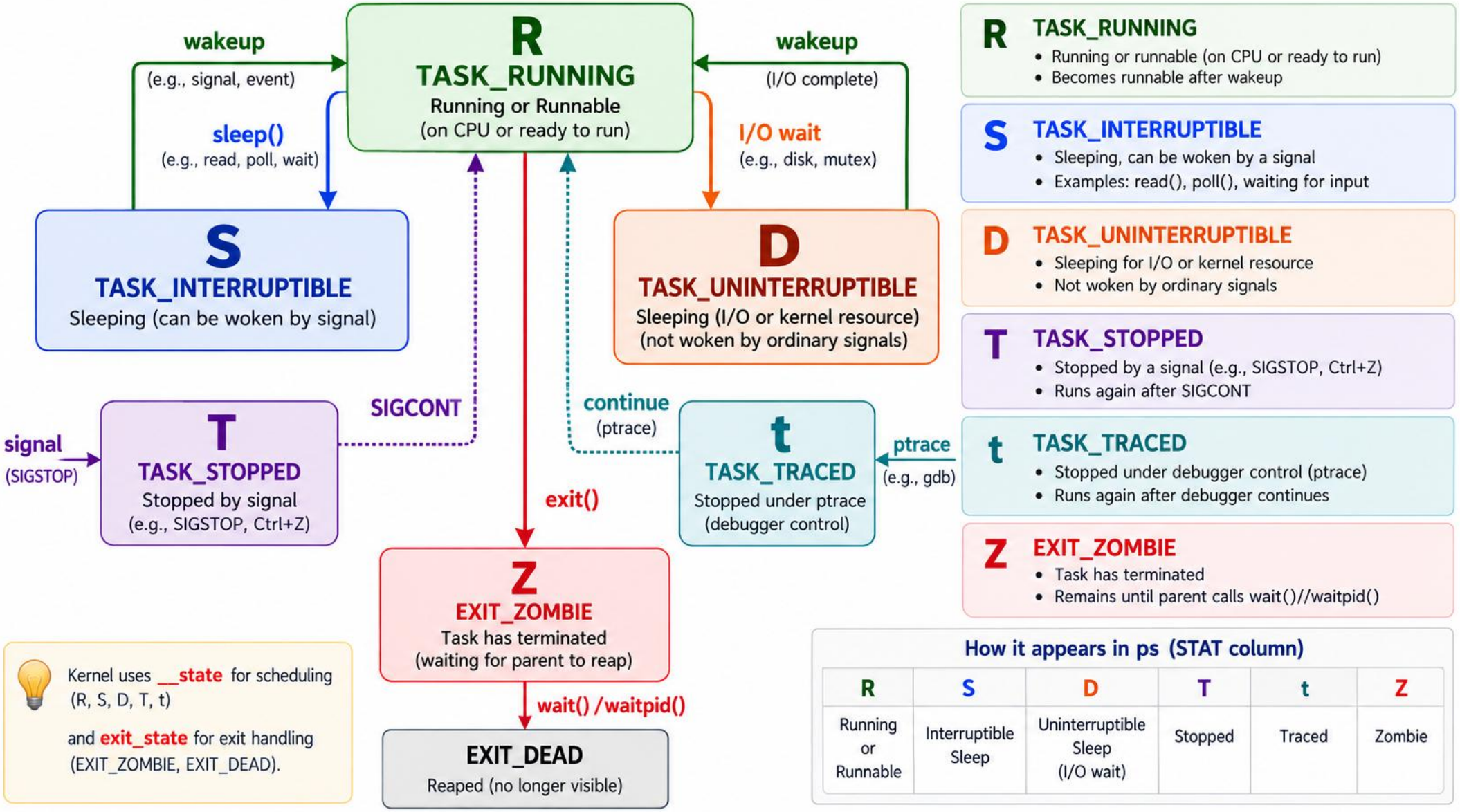
Use the **pstree** command to see the process hierarchy.

Common Use

Helps understand which process started others (e.g., services, shells, applications).

Result: **pstree** shows the **process hierarchy**, from the root process (PID 1) to all **running** processes.

Task States in Linux



AI-generated illustration using ChatGPT (OpenAI)

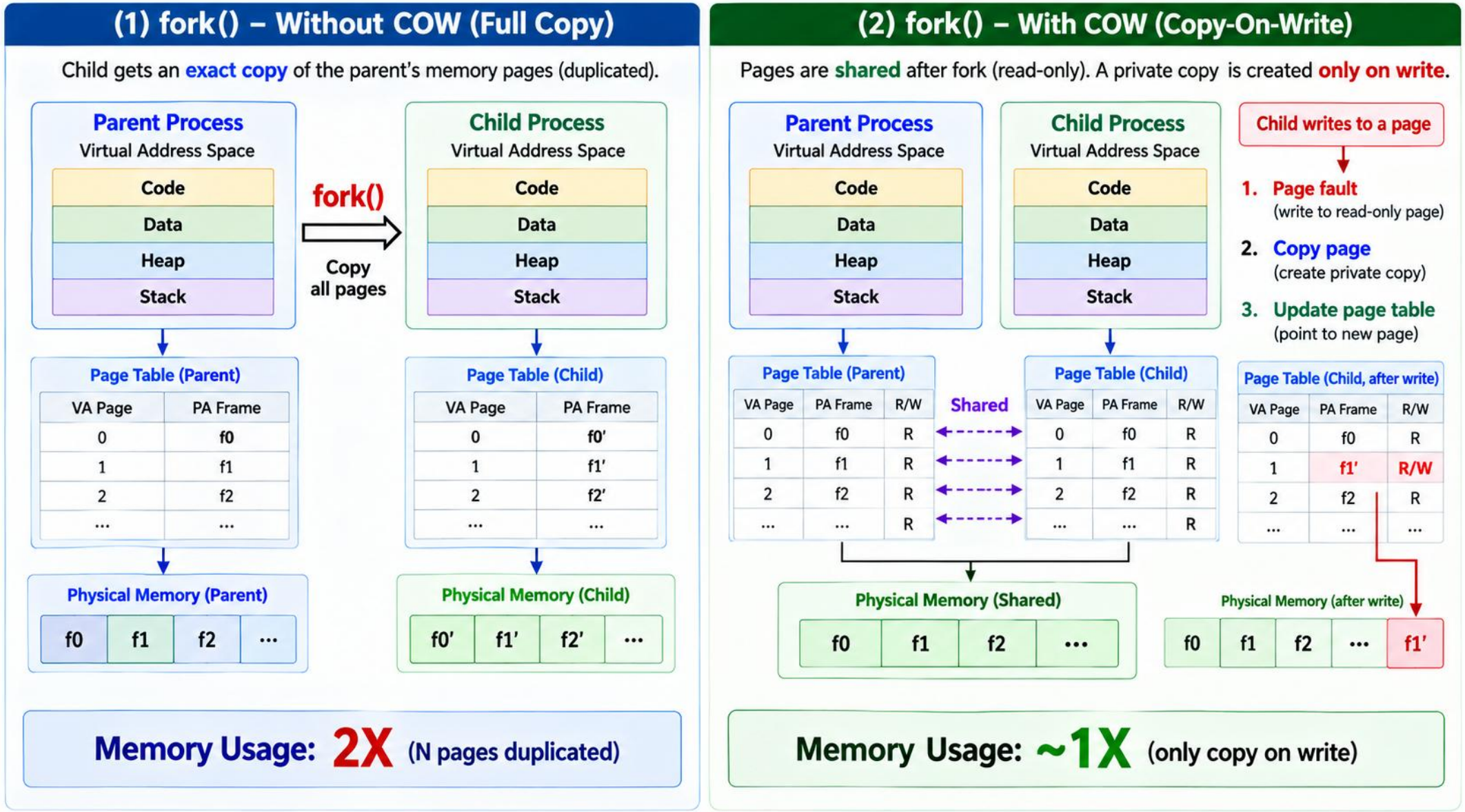
Creating Processes

- ***Parent process*** **creates a new running** ***child process*** **by calling** `fork`

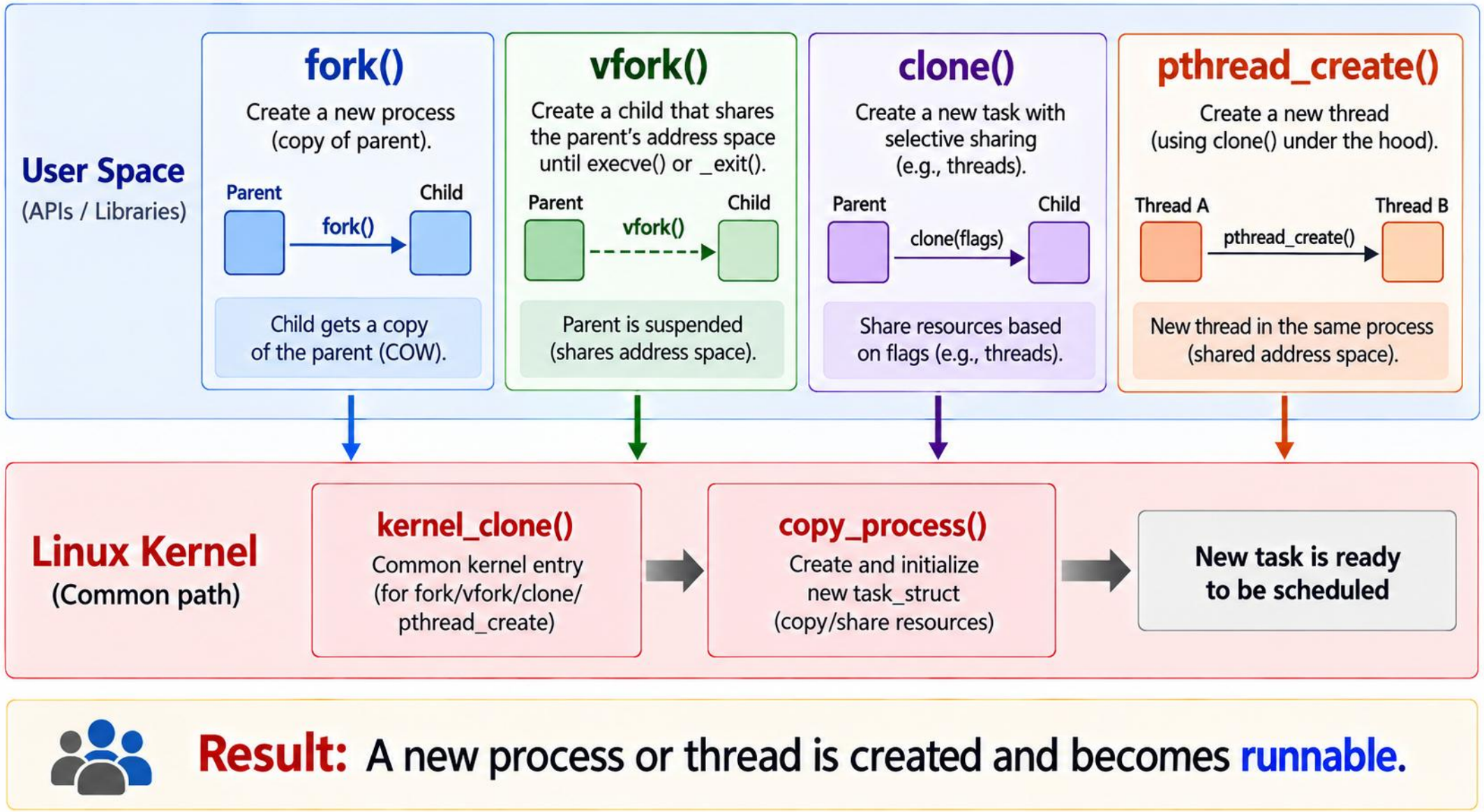
- `int fork(void)`
  - Returns 0 to the child process, child's PID (process id) to parent process
  - Child is *almost* identical to parent:
    - Child gets an identical (but separate) copy of the parent's virtual address space.
    - Child gets identical copies of the parent's open file descriptors
    - Child has a different PID than the parent

- `fork` **is interesting (and often confusing) because it is called** ***once*** **but returns** ***twice***
- **Remember that Linux sees both processes and threads as tasks**

Conceptual View of fork



Four Methods to Create Tasks in Linux



Comparison of Creating Tasks in Linux

Aspect	fork() Create a new process (copy of parent)	vfork() Share parent's address space until exec() or _exit()	clone() Create a task with selective sharing (controlled by flags)	pthread_create() Create a new thread (in the same process)
				
Address Space	Separate (new mm_struct)	Shared (temporary) (same mm_struct)	Depends on flags (e.g., CLONE_VM)	Shared (same mm_struct)
Page Table / Memory	Copied (COW)	Shared (no copy)	Depends on flags (e.g., CLONE_VM)	Shared (no copy)
Copy-On-Write	Yes	No	Depends on flags	No
File Descriptors	Copied (independent table)	Separate descriptor table (entries refer to the same open file descriptions)	Depends on flags (e.g., CLONE_FILES)	Shared (same table)
Execution	Parent and child run concurrently	Parent is suspended until child calls exec() or _exit()	Both run concurrently (according to flags)	All threads run concurrently (in the same process)
Typical Use	Create a new process (general purpose)	fork + exec optimization	Low-level task creation (process or thread)	Create a new thread (multithreading)



Key Point: fork() copies, vfork() shares (until exec), clone() is flexible, pthread_create() creates a thread.

fork Example

```
int main(int argc, char** argv)
{
    pid_t pid;
    int x = 1;

    pid = fork();
    if ( pid == 0 ) {  /* Child */
        printf("child: x=%d\n", ++x);
        exit(0);
    }

    /* Parent */
    printf("parent: x=%d\n", --x);
    exit(0);
}
```

- **Call once, return twice**
- **Concurrent execution**
  - Can't predict execution order of parent and child
- **Duplicate but separate address space**
  - x has a value of 1 when fork returns in parent and child
  - Subsequent changes to x are independent
- **Shared open files**
  - stdout is the same in both parent and child

```
linux> ./fork
parent: x=0
child: x=2
```

```
linux> ./fork
child: x=2
parent: x=0
```

```
linux> ./fork
parent: x=0
child: x=2
```

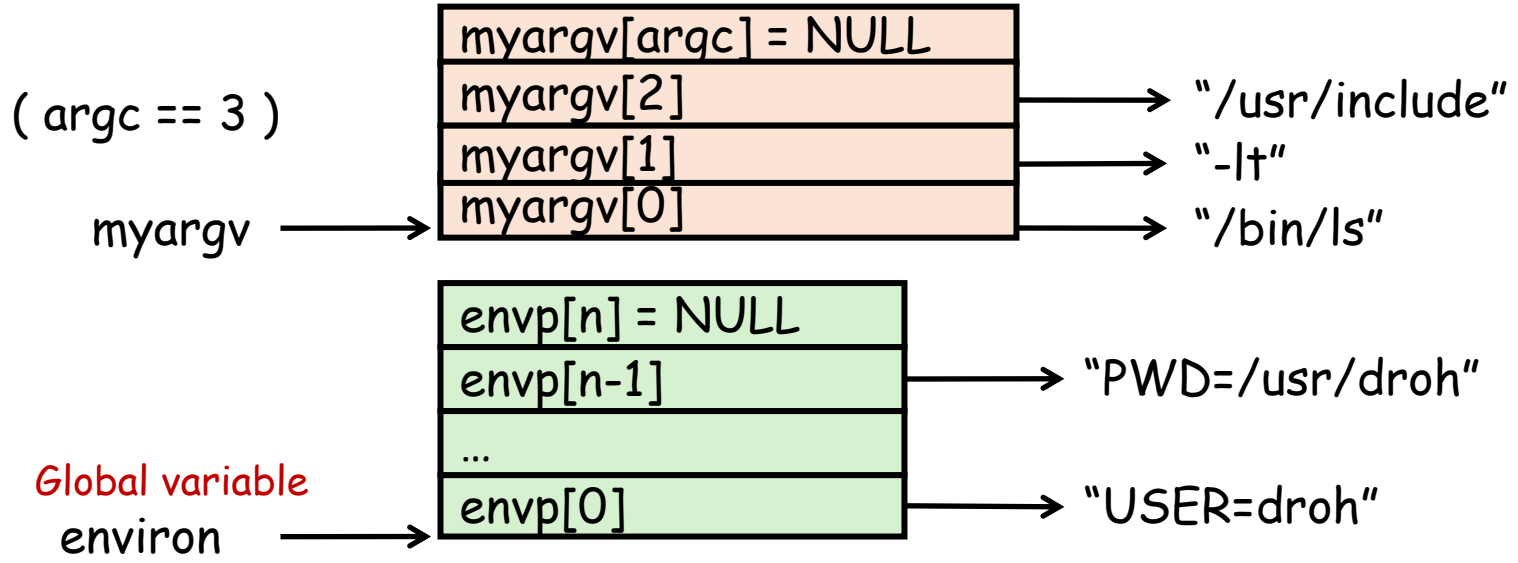
```
linux> ./fork
parent: x=0
child: x=2
```

execve: Loading and Running Programs

- **int execve(char *filename, char *argv[], char *envp[])**
- **Loads and runs in the current process:**
  - Executable file **filename**
    - Can be object file or script file beginning with #!interpreter (e.g., #!/bin/bash)
  - …with argument list **argv**
    - By convention **argv[0]==filename**
  - …and environment variable list **envp**
    - "name=value" strings (e.g., USER=droh)
    - getenv, putenv, printenv
- **Overwrites code, data, and stack**
  - Retains PID and open file descriptors (except close-on-exec FDs (FD_CLOEXEC))
  - Replaces the address space; caught signal handlers are reset
- **Called once and never returns**
  - …except if there is an error

execve Example

- **Executes "/bin/ls -lt /usr/include" in child process using current environment:**




```
if ( ( pid = fork() ) == 0 ) {    /* Child runs program */
        if ( execve(myargv[0], myargv, environ) < 0 ) {
            printf("%s: Command not found.\n", myargv[0]);
            exit(1);
        }
}
```

Terminating Processes

- **Process becomes terminated for one of three reasons:**
  - Calling the `exit` function
  - Returning from the `main` routine
  - Receiving a signal whose default action is to terminate (will be covered later)

- **`void exit(int status)`**
  - Terminates with an *exit status* of `status`
  - Convention: normal return status is 0, nonzero on error
  - Another way to explicitly set the exit status is to return an integer value from the main routine

- **`exit` is called once but never returns.**

Reaping Child Processes

- **When process terminates, it still consumes system resources**
- **UNIX-based OS requires parent process to reap child processes using wait or waitpid**
- **What if parent doesn't reap?**
  - A child process is terminated, but its parent has not yet reaped the child via wait or waitpid -> child process becomes a **zombie** process
    - A zombie (or defunct) process is a process that has completed execution but still has an entry in the process table
    - When the parent calls wait or waitpid, the zombie is reaped and its remaining kernel bookkeeping is released
    - All processes can normally exist as zombie processes only briefly
  - When a parent didn't invoke wait and instead terminated, its children processes are called **orphan** processes
    - An orphan is reparented and eventually reaped by an appropriate reaper (e.g., *init* or *systemd*)

Zombie Example

```
linux> ./forks 7 &
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6639 ttyp9    00:00:03 forks
 6640 ttyp9    00:00:00 forks <defunct>
 6641 ttyp9    00:00:00 ps
linux> kill 6639
[1]    Terminated
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6642 ttyp9    00:00:00 ps
```

```
void fork7() {
    if ( fork() == 0 ) { /* Child */
        printf("Terminating Child, PID = %d\n", getpid());
        exit(0);
    } else { /* Parent */
        printf("Running Parent, PID = %d\n", getpid());
        while (1) ; /* Infinite loop */
    }
}
```

- **ps** shows child process as “defunct” (i.e., a zombie)
- Killing parent allows child to be reparented and reaped by an appropriate reaper (e.g., *init* or *systemd*)

Non-terminating Child Example

```
linux> ./forks 8
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6676 ttyp9    00:00:06 forks
 6677 ttyp9    00:00:00 ps
linux> kill 6676
linux> ps
  PID TTY          TIME CMD
 6585 ttyp9    00:00:00 tcsh
 6678 ttyp9    00:00:00 ps
```

```
void fork8()
{
    if ( fork() == 0 ) { /* Child */
        printf("Running Child, PID = %d\n", getpid());
        while (1) ; /* Infinite loop */
    } else { /* Parent */
        printf("Terminating Parent, PID = %d\n", getpid());
        exit(0);
    }
}
```

- Child process still active even though parent has terminated
- Must kill child explicitly, or else will keep running indefinitely

wait: Synchronizing with Children

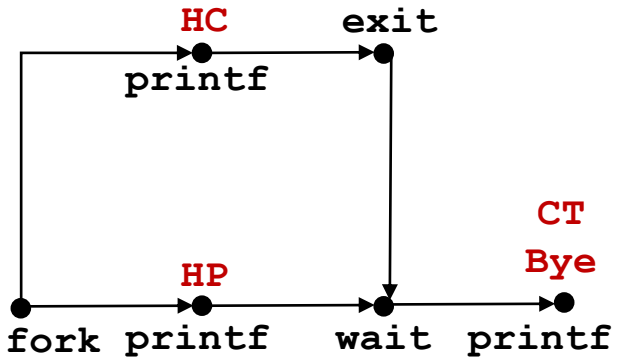
- **Parent reaps a child by calling the wait function**

- **int wait(int *child_status)**
  - Suspends current process until one of its children terminates
  - Return value is the **pid** of the child process that terminated
  - If **child_status != NULL**, then the integer it points to will be set to a value that indicates reason the child terminated and the exit status:
    - Checked using macros defined in wait.h
      - WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG, WIFCONTINUED
      - See man page for details

Synchronizing with Children Example

```
void fork9() {
    int child_status;

    if (fork() == 0) { /* Child */
        printf("HC: hello from child\n");
        exit(0);
    } else { /* Parent */
        printf("HP: hello from parent\n");
        wait(&child_status);
        printf("CT: child has terminated\n");
    }
    printf("Bye\n");
}
```



Feasible output:
HC
HP
CT
Bye

Infeasible output:
HP
CT
Bye
HC

Another wait Example

- If multiple children completed, will take in arbitrary order
- Can use macros WIFEXITED and WEXITSTATUS to get information about exit status

```
void fork10() {
    pid_t pid[N];
    int i, child_status;

    for ( i = 0; i < N; i++ )
        if ( ( pid[i] = fork() ) == 0 ) {
            exit(100+i); /* Child */
        }
    for ( i = 0; i < N; i++ ) { /* Parent */
        pid_t wpid = wait(&child_status);
        if ( WIFEXITED(child_status) )
            printf("Child %d terminated with exit status %d\n", wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

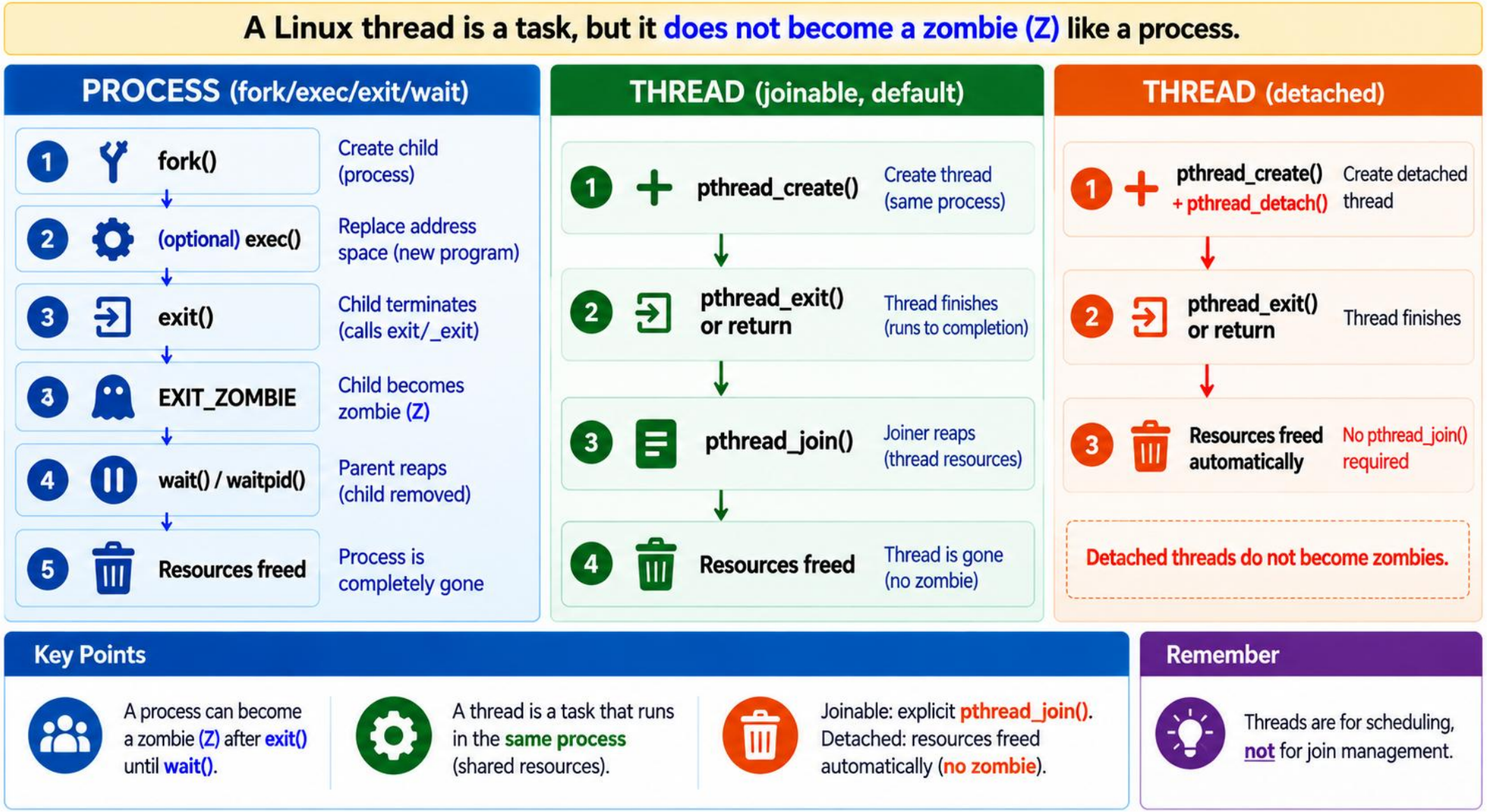
waitpid: Waiting for a Specific Process

- **pid_t waitpid(pid_t pid, int *status, int options)**
  - Suspends current process until specific process (*if pid > 0*) terminates; *if pid == -1*, suspend until any of the parent's child processes terminates (same as wait(&status))
  - Various options (see man page)

```
void fork11() {
    pid_t pid[N];
    int i, child_status;

    for ( i = 0; i < N; i++ )
        if ( ( pid[i] = fork() ) == 0 ) exit(100+i); /* Child */
    for ( i = N-1; i >= 0; i-- ) {
        pid_t wpid = waitpid(pid[i], &child_status, 0);
        if ( WIFEXITED(child_status) )
            printf("Child %d terminated with exit status %d\n", wpid, WEXITSTATUS(child_status));
        else
            printf("Child %d terminate abnormally\n", wpid);
    }
}
```

Thread Termination and Reaping in Linux

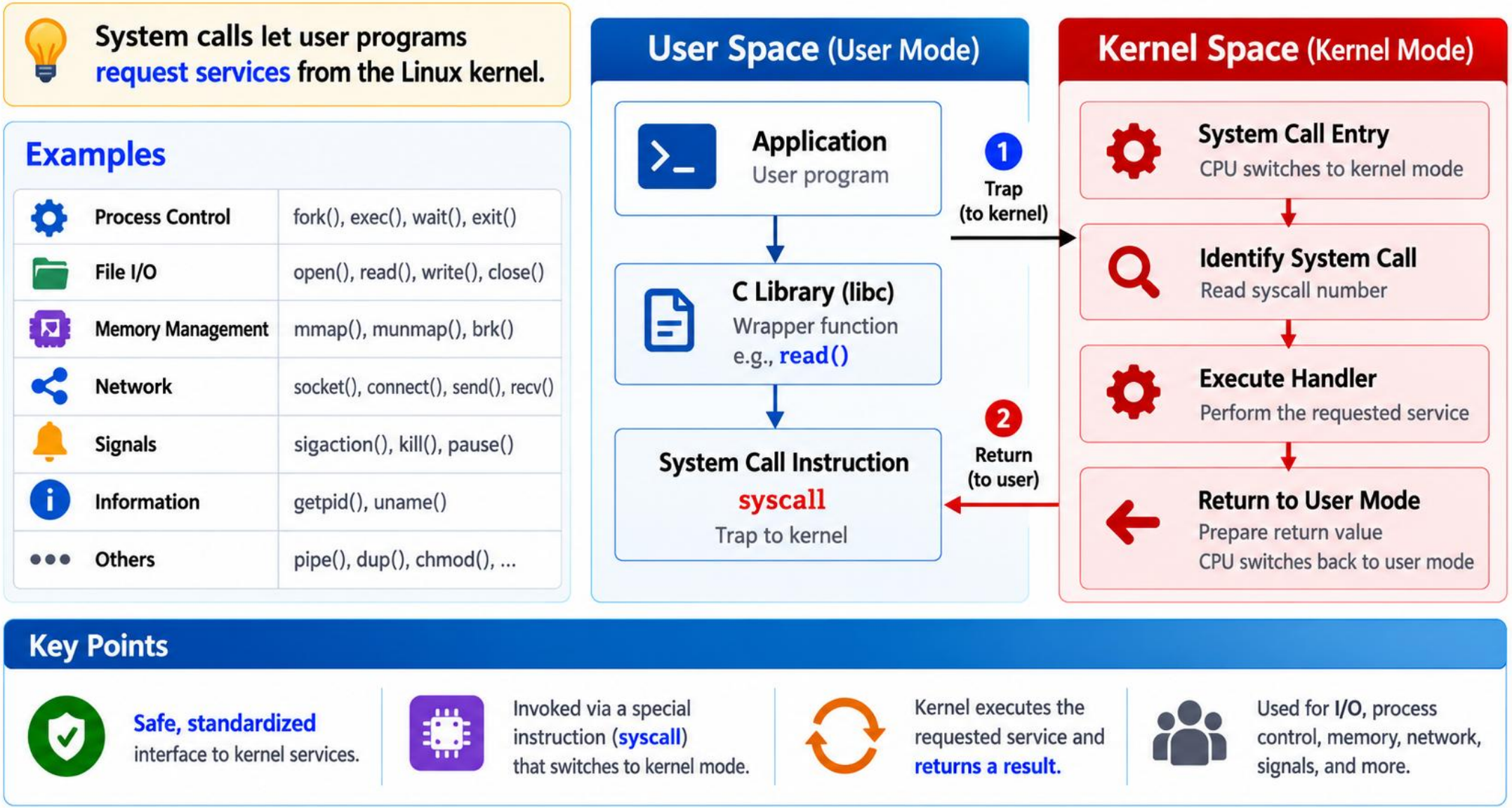


PART 1: Linux Execution Model

- **Week 1**

- **Processes and Threads**
  - How Linux represents and executes tasks
- **System Calls**
  - How programs request services from the kernel
- **File Descriptors**
  - How Linux provides a unified interface to I/O resources

System Calls: The Interface between User Program and Kernel



Parameter Passing: Application Binary Interface (ABI)

1. Normal Function Call (User Space ABI)

Call a C function (e.g., *foo(a, b, c, d, e, f)*)

RDI	arg1
RSI	arg2
RDX	arg3
RCX	arg4
R8	arg5
R9	arg6
stack	arg7, arg8, ...

C function call uses **RCX** for the 4th argument.

If more than 6 arguments, additional arguments are passed on the stack.

Return value: RAX

2. Linux System Call (x86-64)

Invoke a system call (e.g., read(fd, buf, count))

RAX	syscall number (input)
RDI	arg1
RSI	arg2
RDX	arg3
R10	arg4 (not RCX)
R8	arg5
R9	arg6

No stack arguments
(system-call ABI supports at most 6 arguments)

The syscall instruction uses **RCX** internally (to store the user RIP), so Linux uses **R10** for the 4th argument instead of RCX.

System-call arguments are not passed on the user stack.

If more information is needed, pass a pointer to a structure or buffer.

Return value: RAX
(>= 0 on success, < 0 on error: -errno)

3. Example: read() and >6 Arguments

a) read(int fd, void *buf, size_t count)

```
ssize_t n = read(fd, buf, count);
```

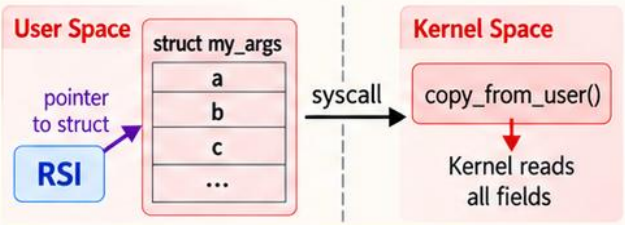
RAX	__NR_read
RDI	fd
RSI	buf
RDX	count
R10	-
R8	-
R9	-

Return value
RAX
n >= 0 : bytes read
n < 0 : -errno

b) More than 6 arguments

Pass a pointer to a structure (or buffer) containing arguments.

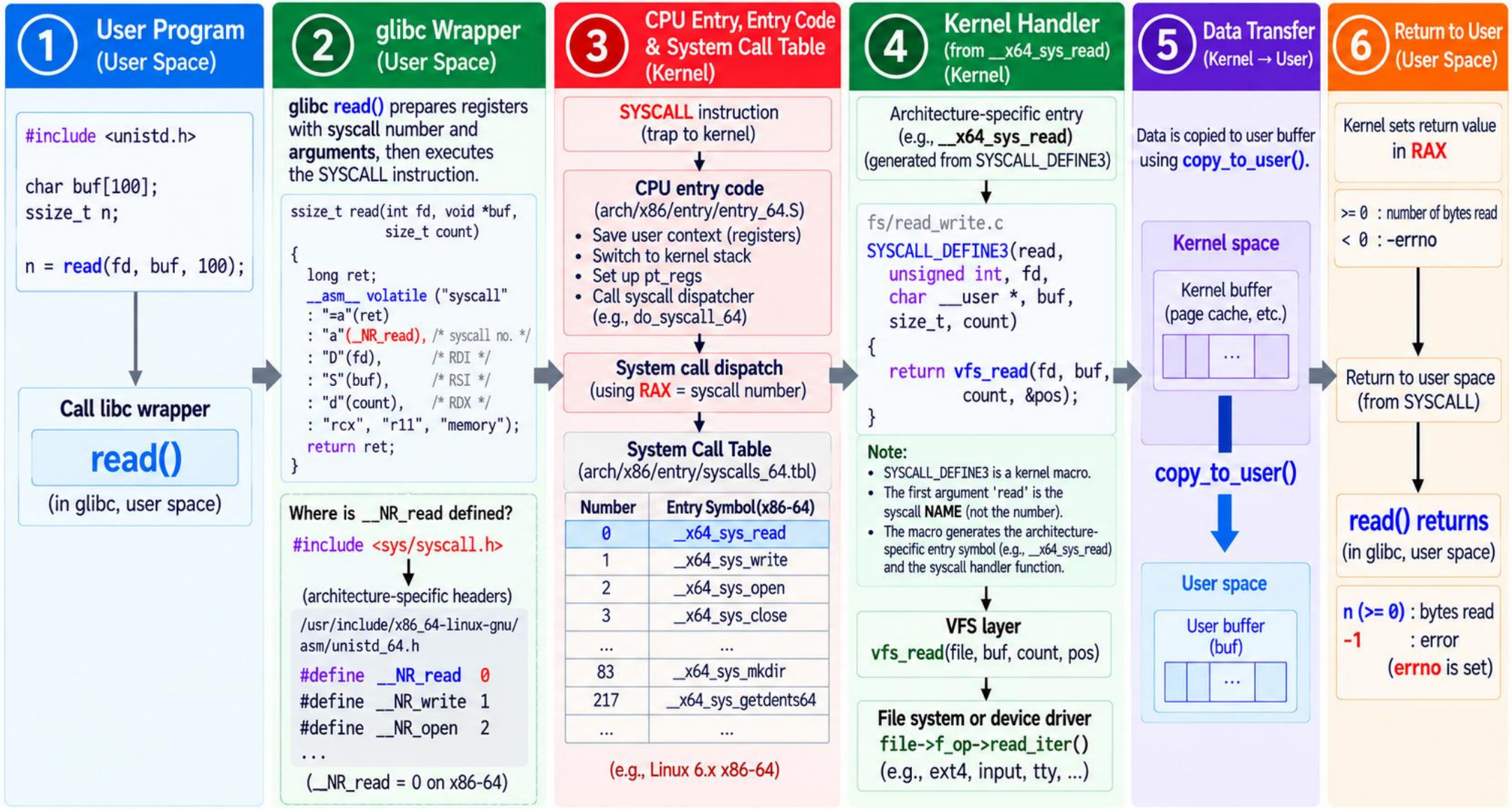
```
struct my_args args = { ... };
my_syscall(fd, &args);
```



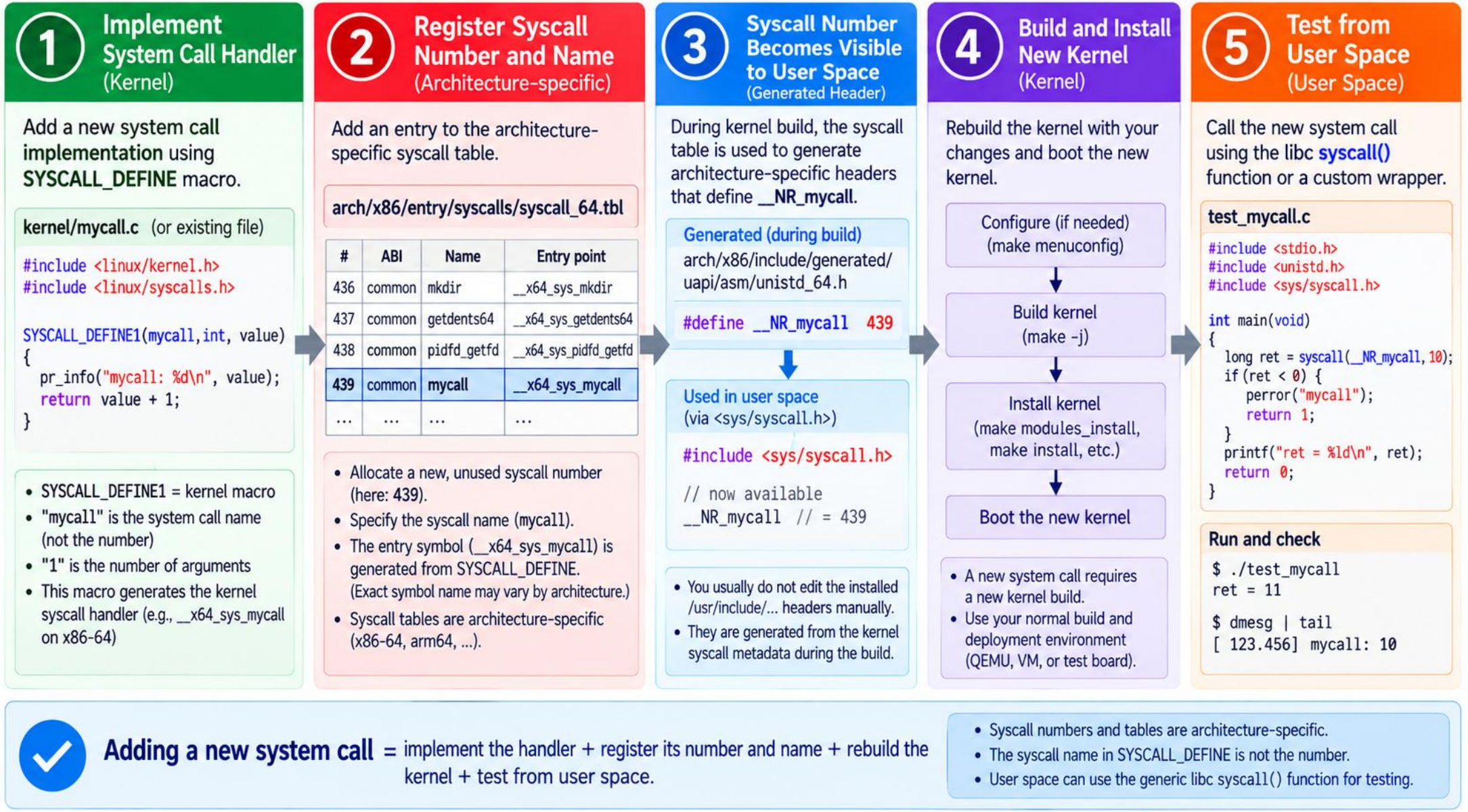
Key Takeaways

- Function calls use **RCX** for 4th arg; system calls use **R10**.
- System-call ABI supports at most 6 arguments (in registers).
- System-call arguments are **not** passed on the user stack.
- Pass a pointer to a structure for more information.

Overall Flow of read() System Call



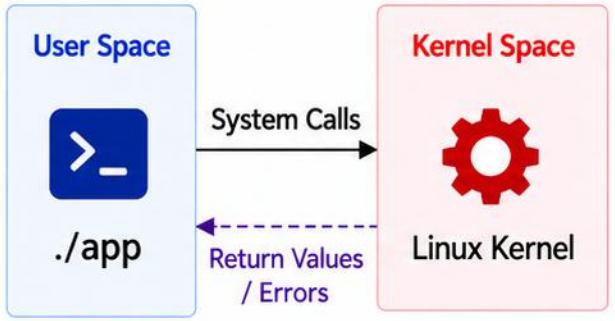
How to Add a New System Call



strace: Observe across System Call Boundary

strace lets you see the system calls made by a process.

How it works



What strace shows

- Which system calls are invoked
- Arguments passed to each call
- Return value (success or error)
- Temporal order and timing

Example: strace ./app

```
$ strace -f -tt -T ./app
```

```
12:34:56.789012  execve("./app", ["./app"], 0x7fff...) = 0            <0.002341>
12:34:56.792345  brk(NULL)                              = 0x55b1a84c000 <0.000023>
12:34:56.792400  mmap(NULL, 8192, PROT_READ|PROT_WRITE,
                     MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f3b2c7e9000 <0.000041>
12:34:56.792512  openat(AT_FDCWD, "data.txt", O_RDONLY) = 3            <0.000056>
12:34:56.792600  read(3, "Hello, Linux!\n", 14)         = 14           <0.000038>
12:34:56.792738  write(1, "Hello, Linux!\n", 14)        = 14           <0.000029>
12:34:56.792854  close(3)                               = 0            <0.000018>
12:34:56.792902  exit_group(0)                          = ?            <0.000031>
```

- **Timestamp** — When the call was made
- **System Call** — Kernel service invoked
- **Arguments** — Parameters passed to the call
- **Return Value** — Result of the call (>= 0: success, < 0: -errno)
- **Time** — Time spent in the call (seconds)

Tip | Useful options: `-f` (follow forks) `-tt` (absolute timestamps) `-T` (show time in calls) `-e trace=...` (filter specific calls)

ftrace: Observe What Happens inside Kernel

ftrace traces kernel execution paths and timing.

It records which kernel functions are executed, when, how long they take, and the call paths **(overhead is configurable)**.

1 Configure ftrace
(enable function tracing)

```
# 1. Go to tracing directory
$ cd /sys/kernel/tracing

# 2. Enable function tracing (clean start)
$ sudo sh -c 'echo 0 > tracing_on'
$ sudo sh -c 'echo function_graph
  > current_tracer'

# 3. (Optional) trace only read() path
$ sudo sh -c 'echo __x64_sys_read
  > set_graph_function'

# 4. Clear old trace
$ sudo sh -c 'echo > trace'
```

2 Run workload
(generate ftrace events)

```
# 1. Enable tracing
$ sudo sh -c 'echo 1 > tracing_on'

# 2. Run a program (calls read())
$ ./app
# e.g., $ cat data.txt

# 3. Stop tracing
$ sudo sh -c 'echo 0 > tracing_on'
```

Example program (app)

```
int fd = open("data.txt", O_RDONLY);
read(fd, buf, sizeof(buf));
close(fd);
```

Kernel executes functions

ftrace records events automatically
(no separate ftrace program)

3 View and understand the result
(function call path and timing)

```
$ cat trace
app-1234  [002]  123.456789:   __x64_sys_read() {
    ├─ ksys_read() {
    │   ├─ vfs_read() {
    │   │   ├─ new_sync_read() {
    │   │   │   ├─ ext4_file_read_iter() {
    │   │   │   │     ...
    │   │   │   }
    │   │   }
    │   }
    }
app-1234  [002]  123.456852:   }  /* return */
```

What the output means

1	app-1234	: process name and PID
2	[002]	: CPU number
3	123.456789	: timestamp (seconds)
4	|— ...	: kernel function call path (nested calls)
5	}	: function return

Key Points

- ftrace shows what the kernel is doing inside (function calls, timing, call paths).
- Use filters (e.g., set_graph_function) to focus on specific functions.
- Overhead depends on the tracer and filters (configurable, not always low).
- Requires root privileges (sudo).

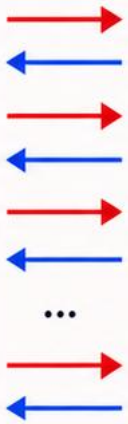
System Calls Are Not Free

Many Small System Calls

User Space

```
for (i = 0; i < N; i++)
  write(fd, &buf[i], 1);
```

N = 100,000



Kernel Space



Validate
Do work
Update state
Return

100,000 writes
→ 100,000 kernel crossings

Each call incurs entry/return + validation + bookkeeping

Measure with perf

How often and where is time spent?

```
$ perf stat -e syscalls:sys_enter_write ./small_writes

syscalls:sys_enter_write   100,000
task-clock                 320 ms
cycles                     1.2 × 109
instructions               2.8 × 109
...
```

High syscall count

```
$ perf record -g ./small_writes
$ perf report

Samples      %    Symbol
1          35.2%  __x64_sys_write
2          18.7%  vfs_write
3          12.1%  ksys_write
4           8.3%  copy_from_user
5           6.1%  entry_SYSCALL_64
...
```

CPU time in syscall path



Flame Graph (hot functions)

Buffer and Batch

Same amount of data, fewer system calls

User Space

```
char buf[64*1024];
prepare_data(buf);
write(fd, buf, 64*1024);
```

Kernel Space



Validate
Do work
Update state
Return

1 write (64 KB)
→ 1 kernel crossing

Amortize the per-call cost



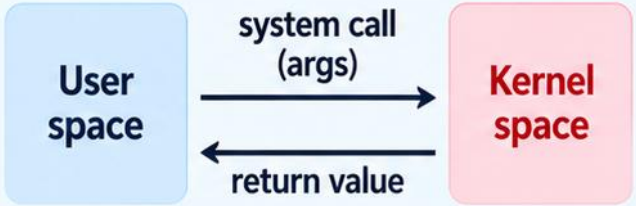
Many tiny system calls can be expensive. → Buffering and batching reduce syscall frequency.

perf helps you **measure** the frequency and cost, not just guess.

Comparison of strace, ftrace, perf

strace

What system calls does the program make?

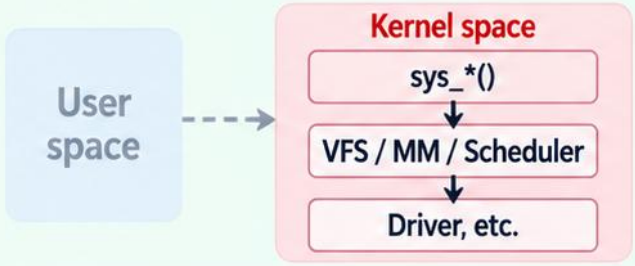


- Shows system call names, arguments, and return values
- Easy to use and low overhead
- Good for functional debugging

Use when You want to know if the program is doing the expected I/O, file, network, or other system calls.

ftrace

What happens inside the kernel?

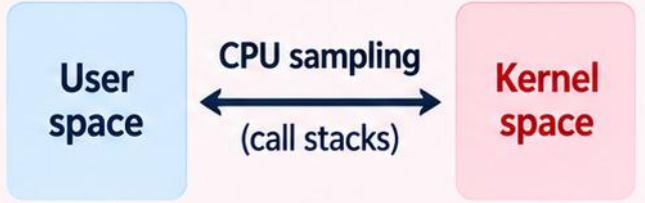


- Traces kernel functions and events (syscalls, scheduler, I/O, interrupts, etc.)
- Shows the execution path and timing
- Good for understanding kernel behavior

Use when You want to see what code runs inside the kernel after a system call, or investigate kernel subsystems.

perf

Where does CPU time go?



- Profiles CPU usage (sampling)
- Shows hotspots with call stacks (user + kernel)
- Supports many hardware counters (cycles, instructions, cache misses, etc.)

Use when You want to find performance bottlenecks and see where CPU time is spent in your application or the kernel.

Choose the tool based on the question, not the tool itself.

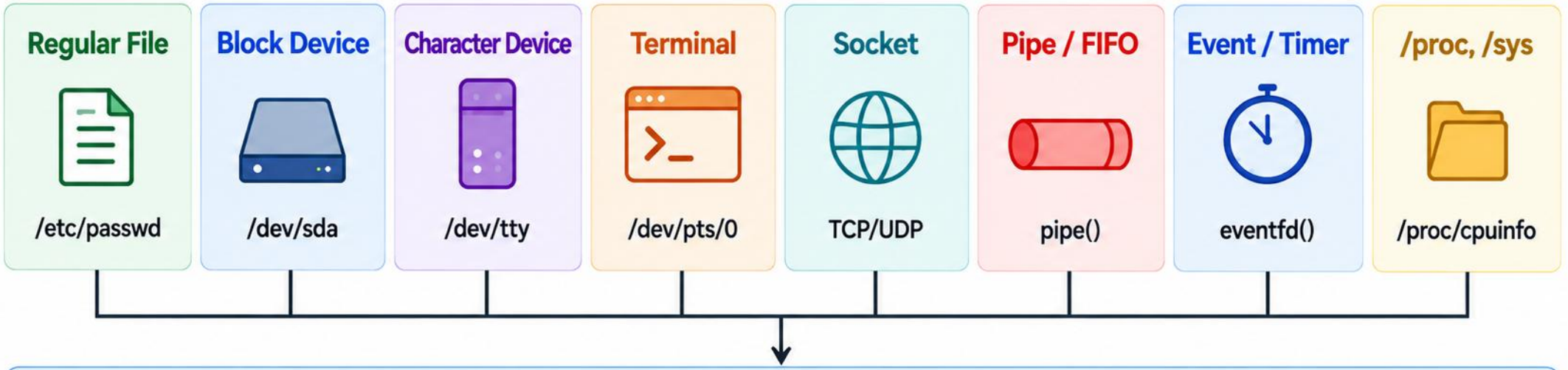
PART 1: Linux Execution Model

- **Week 1**

- **Processes and Threads**
  - How Linux represents and executes tasks
- **System Calls**
  - How programs request services from the kernel
- **File Descriptors**
  - How Linux provides a unified interface to I/O resources

Unified I/O Interface in Unix/Linux

- Unix/Linux provides a **uniform interface for I/O**
- Many different objects can be accessed through file-like interfaces → **file descriptor (FD)**



Unix I/O Interface (via file descriptor)

`open()` • `read()` • `write()` • `lseek()` • `ioctl()` • `close()`

Many I/O objects share a common FD-based interface; supported operations depend on the object type.

Keep in mind:

- Not all objects support every operation.
- The available operations vary by object type (e.g., pipes do not support lseek()).

What is a File Descriptor (FD)?

A **file descriptor (FD)** is a small integer used by a process to refer to an **open object.**

Inside a Process

File Descriptor Table



fd = 3

Opened Object



data.txt

Could be a file, device, socket, pipe, etc.

Example (C)

```c
int fd = open("data.txt", O_RDONLY);
char buf[100];
read(fd, buf, 100);
close(fd);
```

Standard File Descriptors

0	stdin	(standard input)
1	stdout	(standard output)
2	stderr	(standard error)
3, 4, ...	other opened objects	

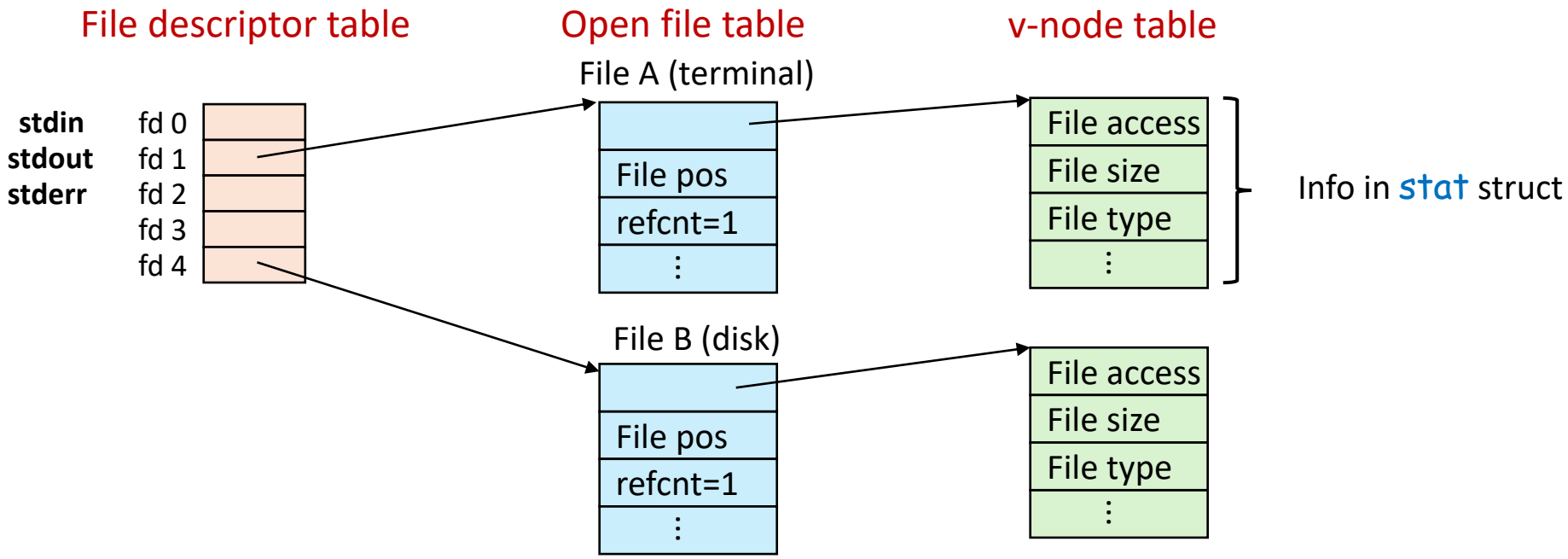


An FD is an index into the process's file descriptor table.

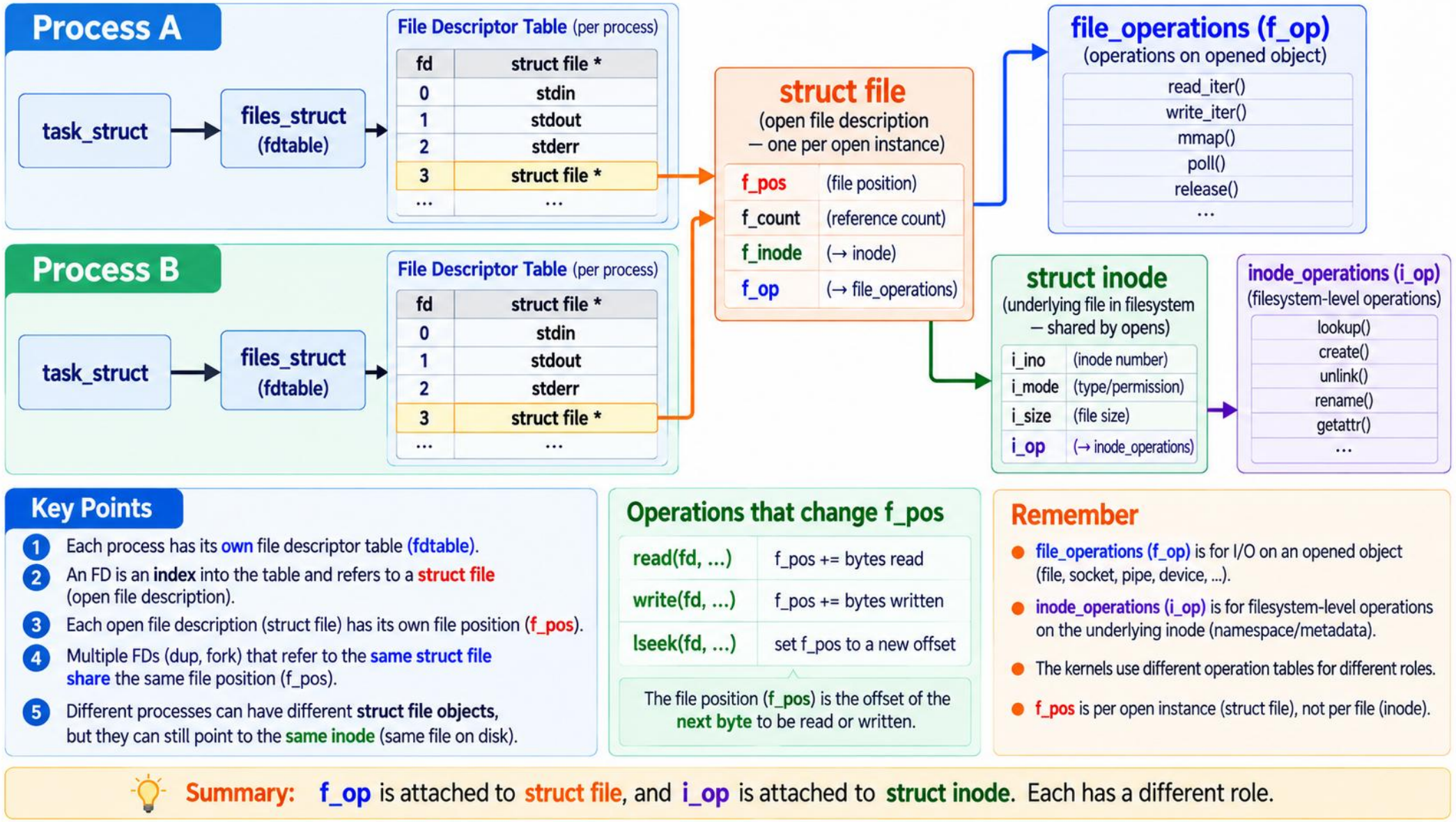
The FD is a **handle** — not the file itself.

Conceptual Kernel Structures for Open Files

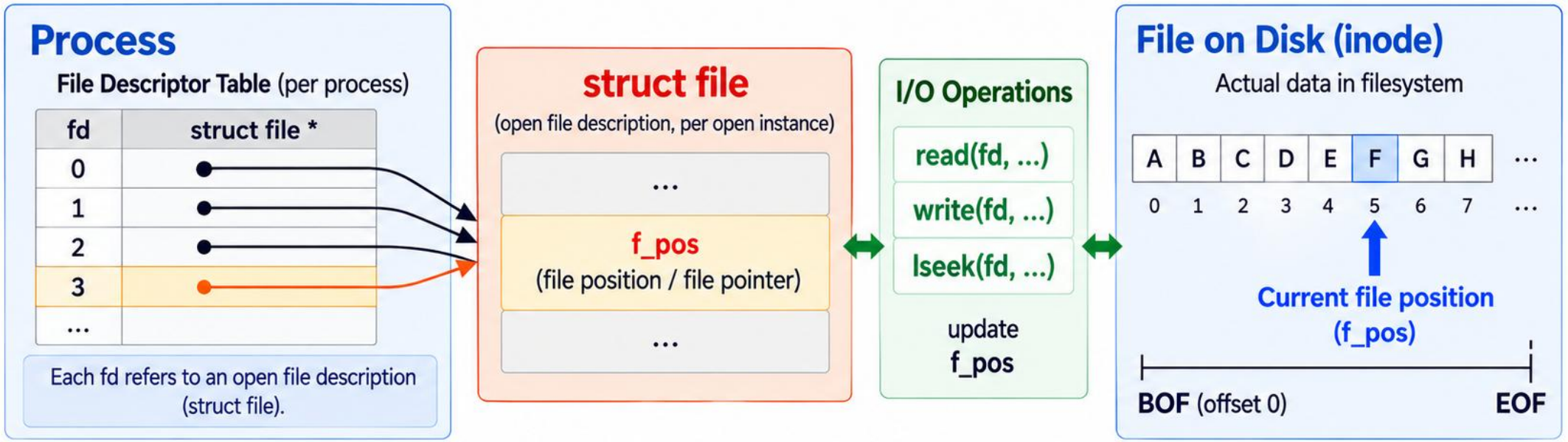
- **File Descriptor Table (one per process)** → just a pointer array
  - Indexed by the **open file descriptors** and each entry has a **pointer** to open file table structure
- **Open File Table (shared by all processes)** → state for opened file
  - Has current **file position**, **reference count**, and a **pointer** to v-node table structure
- **V-node Table (shared by all processes)** → metadata for actual file
  - Contains most of the **file metadata** information in the `stat` structure



File Descriptors in Linux



File Pointer



Key Points

- Each **open file description** (struct file) has its **own file position (f_pos)**.
- All read(), write(), and lseek() share and update the **same f_pos**.
- Multiple file descriptors (dup, fork) that refer to the same open file description also **share the same file position**.

The file position is per open instance, not per file (inode).

Operations that change f_pos

read(fd, ...)	f_pos += bytes read
write(fd, ...)	f_pos += bytes written
lseek(fd, ...)	set f_pos to a new offset

The current file position (f_pos) is the offset of the **next byte** to be read or written.

End of File (EOF)

- EOF is the position just after the last byte in a file.
- Reading at EOF returns 0 (no more data).
- Writing at EOF extends the file (the file grows).

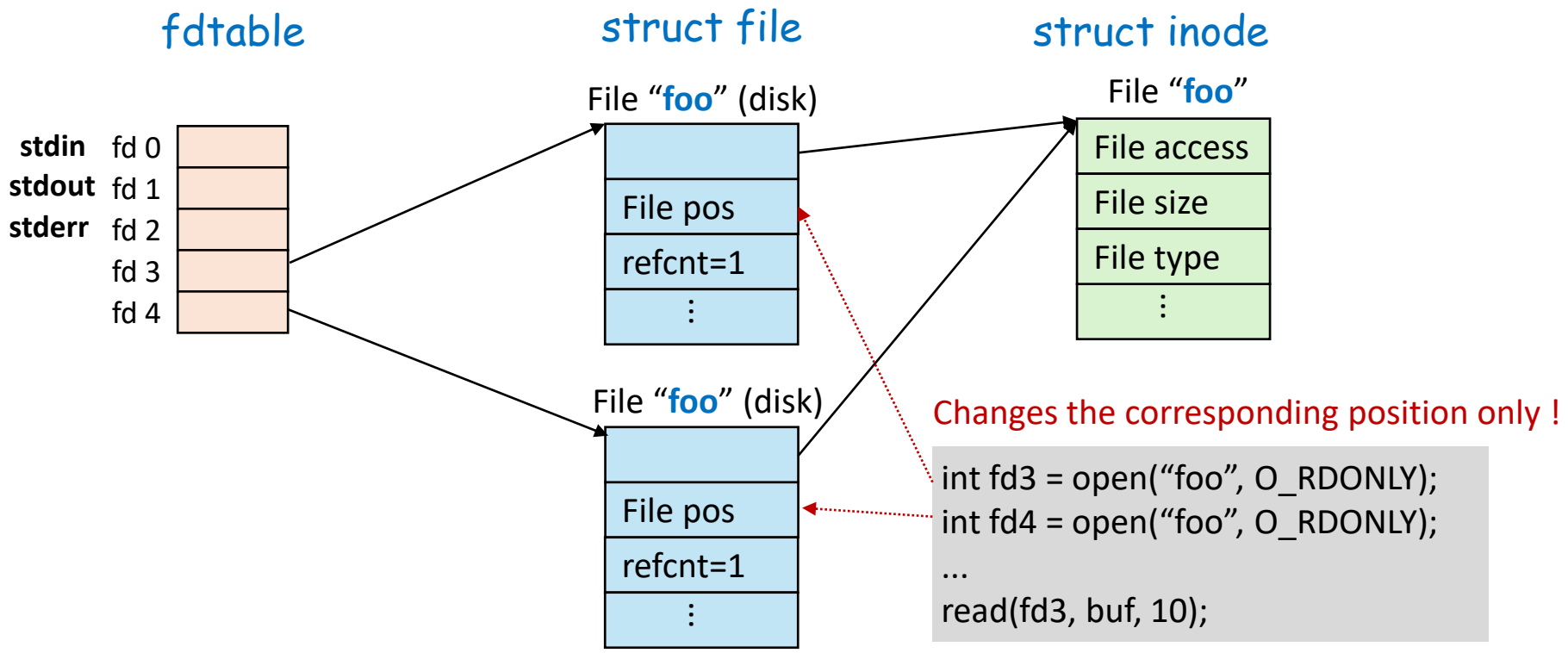
EOF is a position (offset), not an error.

Short Counts

- **Short counts are uncommon for regular disk files**
  - Reading from regular files from disks (except for EOF)
  - Writing data to regular files in disks
- **They commonly occur with:**
  - Encountering (end-of-file) EOF on reads
  - Reading text lines from a terminal
  - Reading from and writing to slow devices
    - I/O operations on pipes, sockets, and terminals may be interrupted by signals (EINTR)
- **Whether a read/write can be interrupted depends on the type of the underlying file or device**
- **Always check the return value of `read()` and `write()`**

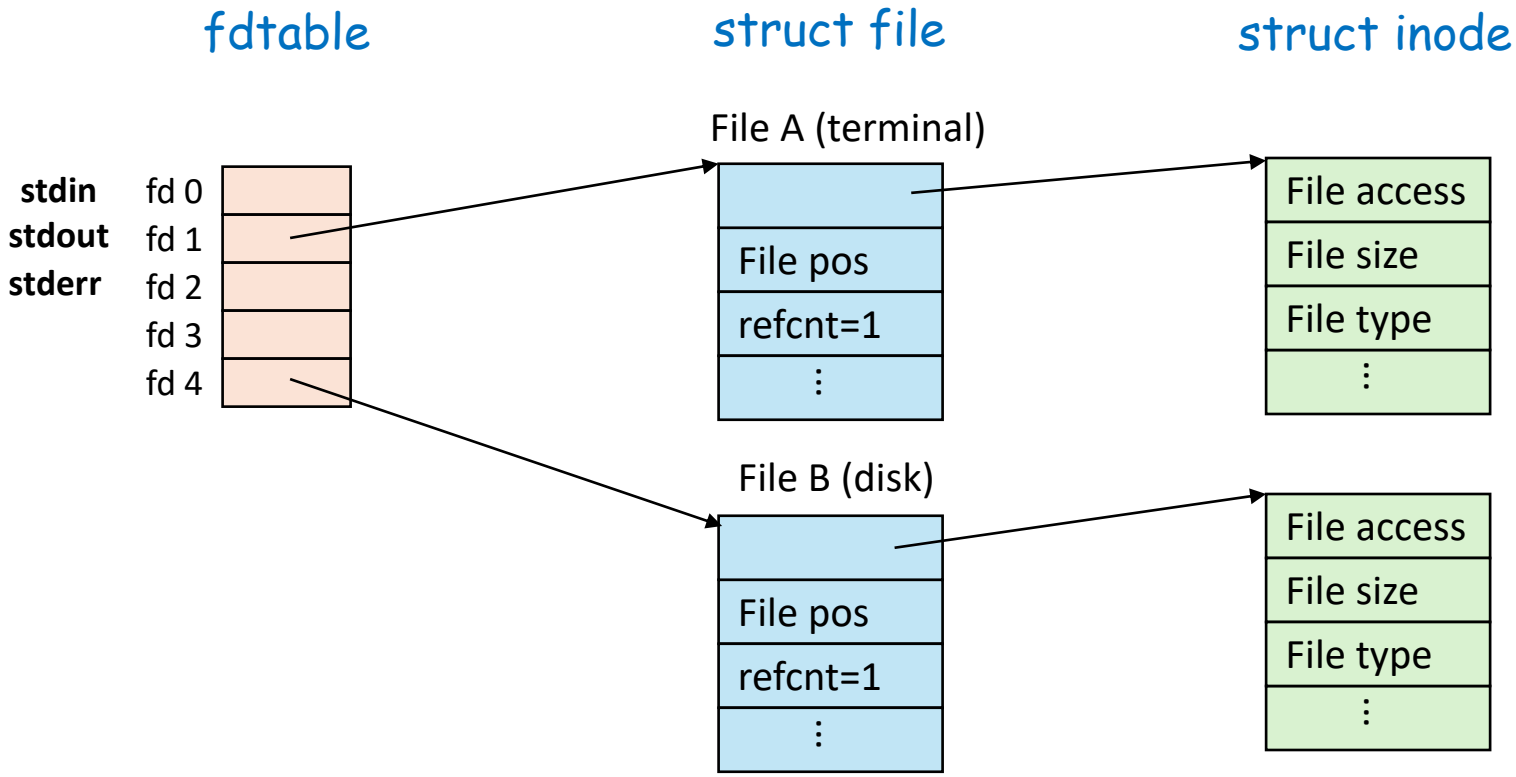
Sharing Disk File

- **Two distinct descriptors sharing the same disk file through two distinct open file table entries**
  - E.g., Calling `open` twice with the same `filename` argument
  - Each entry in the open file table is created on every open



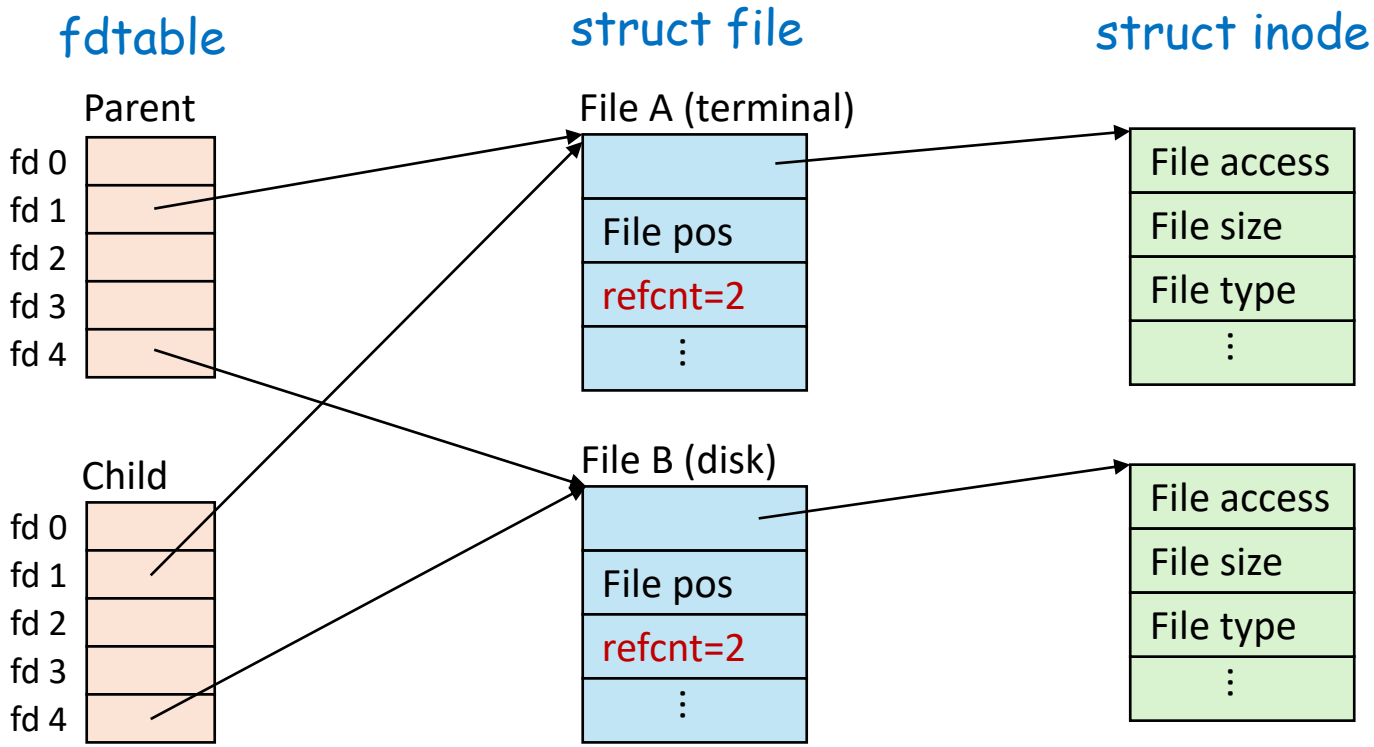
Sharing Files via fork (Before fork)

- *Before* fork call:



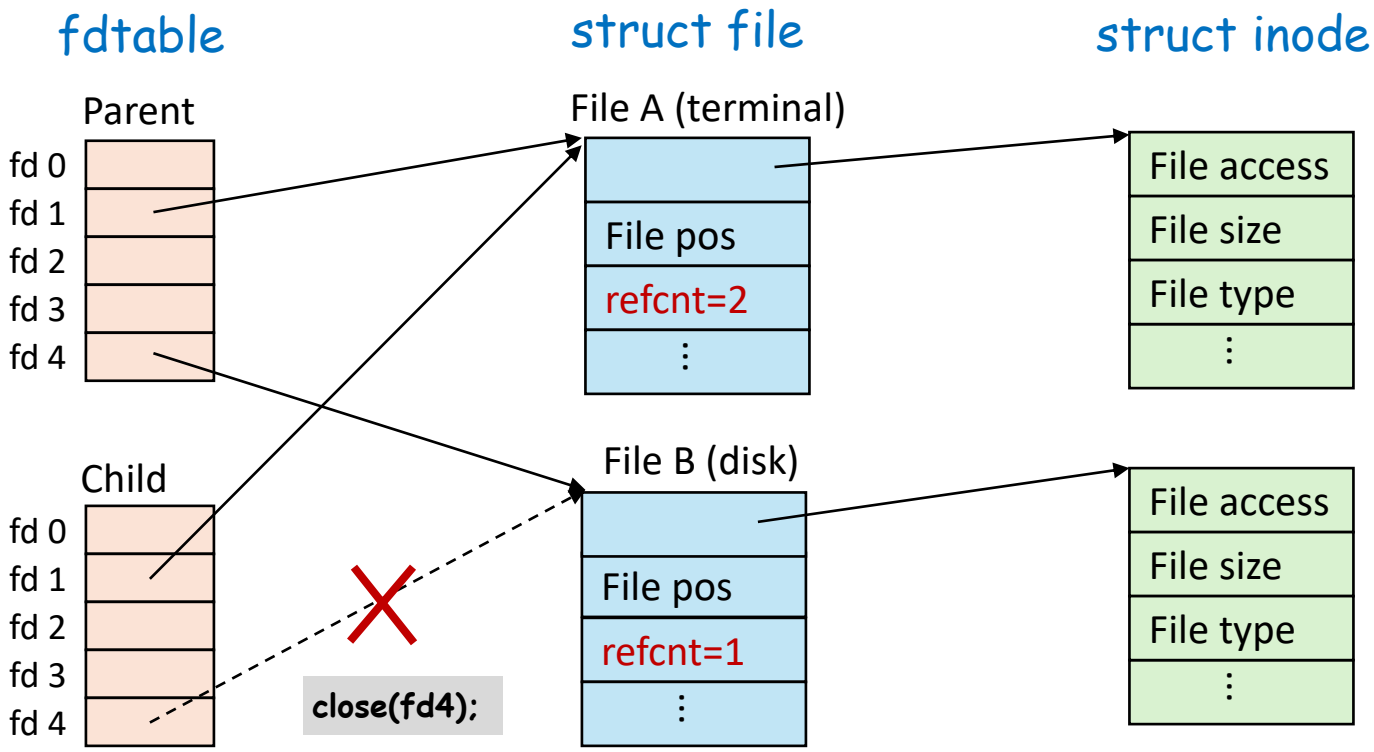
Sharing Files via fork (After fork)

- ***After* fork call:**
  - A child process inherits its parent's open files
  - The child gets its own file descriptor table
  - Corresponding descriptors refer to the same open file descriptors (struct file), increasing its reference count (f_count)



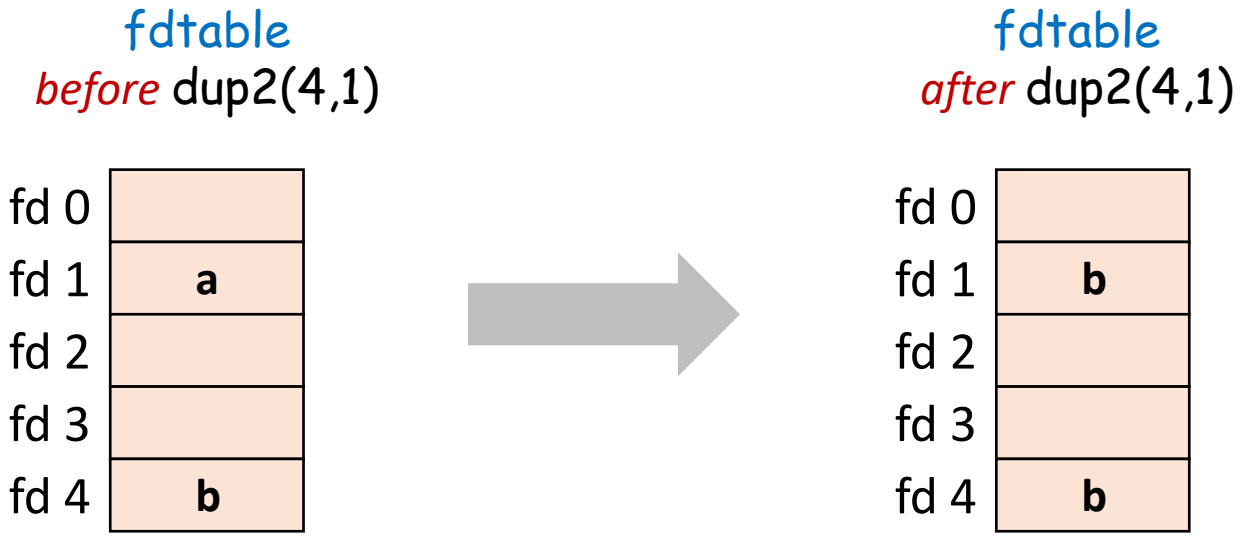
Close Semantics

- **close(fd):**
  - Remove `fd` from the descriptor table
  - Decrement `refcnt` (`f_count`) in the open file table entry
  - If `refcnt == 0`, release open file table entry (possibly release v-node)



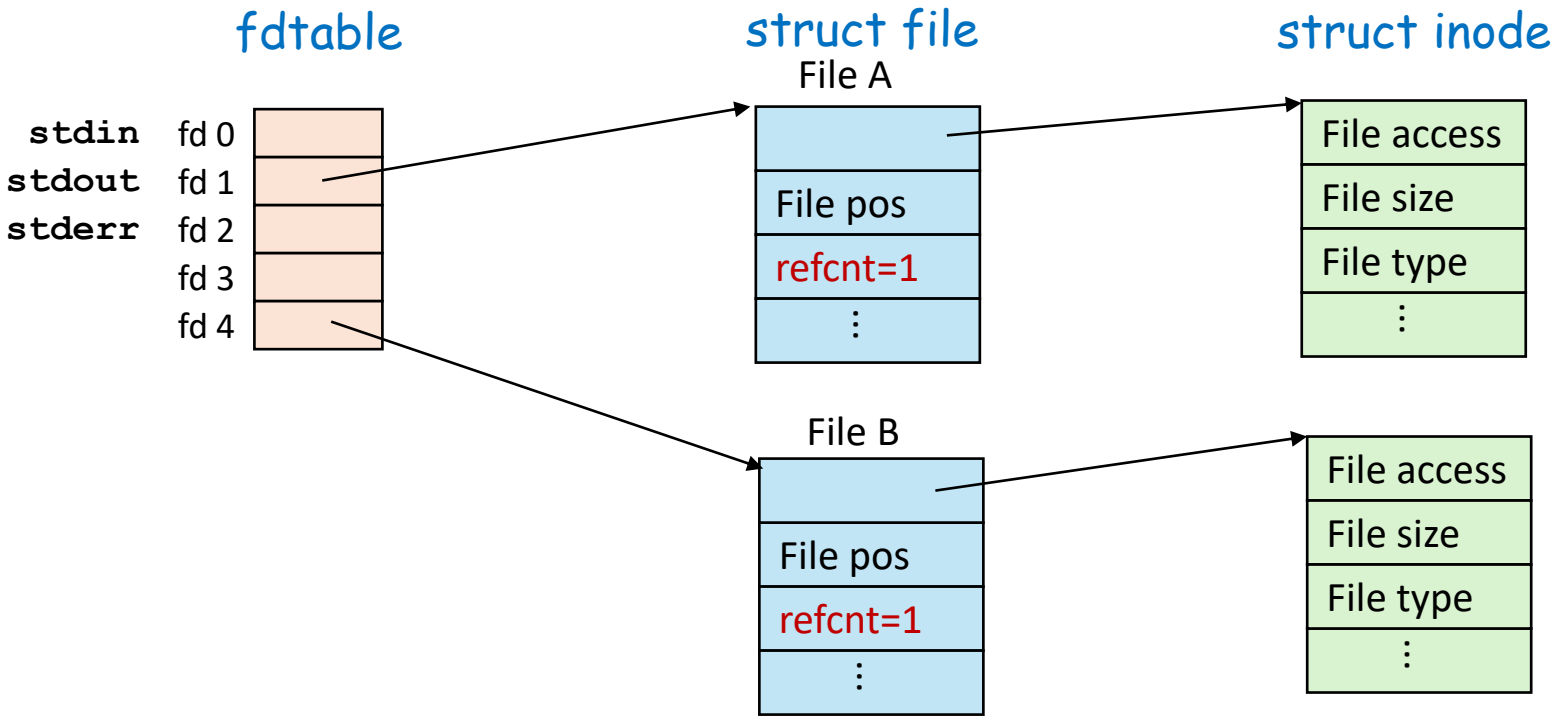
I/O Redirection

- **I/O redirection allows users to associate standard input and output with disk files**
  - ls > foo.txt
- **Question: How does a shell implement I/O redirection?**
- **Answer: By calling the dup2(oldfd, newfd) function**
  - Copies (per-process) descriptor table entry oldfd to entry newfd



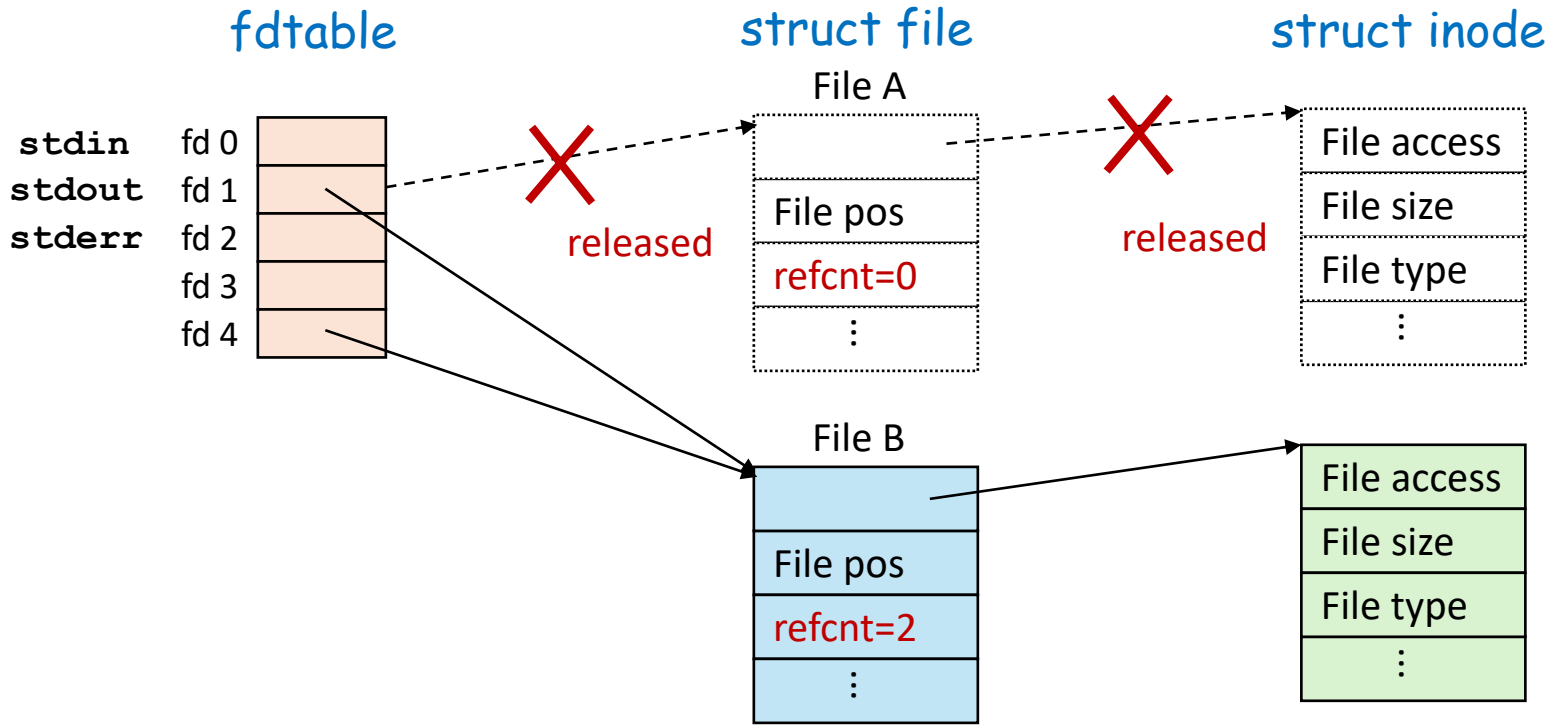
I/O Redirection via dup2 (Before dup2)

- **Step #1: open file to which stdout should be redirected**
  - Happens in child executing shell code, before *exec*
    - `ls > foo.txt` causes shell to load and execute the **ls** program, with standard output redirected to disk file `foo.txt`



I/O Redirection via dup2 (After dup2)

- **Step #2: call dup2(4, 1)**
  - causes fd = 1 (stdout) to refer to disk file pointed at by fd = 4



Example 1: I/O and Redirection

- **What would this program print for file containing "abcde"?**

```
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1, fd2, fd3;
    char c1, c2, c3;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    fd2 = open(fname, O_RDONLY, 0);
    fd3 = open(fname, O_RDONLY, 0);
    dup2(fd2, fd3);
    read(fd1, &c1, 1);
    read(fd2, &c2, 1);
    read(fd3, &c3, 1);
    printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
    return 0;
}
```

Example 1: I/O and Redirection (Output)

- **What would this program print for file containing "abcde"?**

```
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1, fd2, fd3;
    char c1, c2, c3;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    fd2 = open(fname, O_RDONLY, 0);
    fd3 = open(fname, O_RDONLY, 0);
    dup2(fd2, fd3);
    read(fd1, &c1, 1);
    read(fd2, &c2, 1);
    read(fd3, &c3, 1);
    printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
    return 0;
}
```

c1 = a, c2 = a, c3 = b

dup2(oldfd, newfd)

Example 2: Process Control and I/O

- What would this program print for file containing "abcde"?

```
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1;
    int s = getpid() & 0x1;
    char c1, c2;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    read(fd1, &c1, 1);
    if (fork()) { /* Parent */
        sleep(s);
        read(fd1, &c2, 1);
        printf("Parent: c1 = %c, c2 = %c\n", c1, c2);
    } else { /* Child */
        sleep(1-s);
        read(fd1, &c2, 1);
        printf("Child: c1 = %c, c2 = %c\n", c1, c2);
    }
    return 0;
}
```

Example 2: Process Control and I/O (Output)

- **What would this program print for file containing "abcde"?**

```
#include "csapp.h"
int main(int argc, char *argv[])
{
    int fd1;
    int s = getpid() & 0x1;
    char c1, c2;
    char *fname = argv[1];
    fd1 = open(fname, O_RDONLY, 0);
    read(fd1, &c1, 1);
    if (fork()) { /* Parent */
        sleep(s);
        read(fd1, &c2, 1);
        printf("Parent: c1 = %c, c2 = %c\n", c1, c2);
    } else { /* Child */
        sleep(1-s);
        read(fd1, &c2, 1);
        printf("Child: c1 = %c, c2 = %c\n", c1, c2);
    }
    return 0;
}
```

Child: c1 = a, c2 = b
Parent: c1 = a, c2 = c

Parent: c1 = a, c2 = b
Child: c1 = a, c2 = c

Which way does it go?

Standard I/O and Buffering

Standard I/O (stdio)



- High-level I/O library in user space.
- Provides convenient functions:
  fopen(), fread(), fwrite(), fprintf(), ...
- Uses **buffering** to reduce system calls.

Buffering Modes (setbuf / setvbuf)

Fully Buffered



- Write when buffer is full or flushed.
- Used for regular files.

Line Buffered



- Write on newline (\n) or when flushed.
- Used for terminals (TTY).

Unbuffered



- Write immediately (no buffer).
- Buffer size is 0.

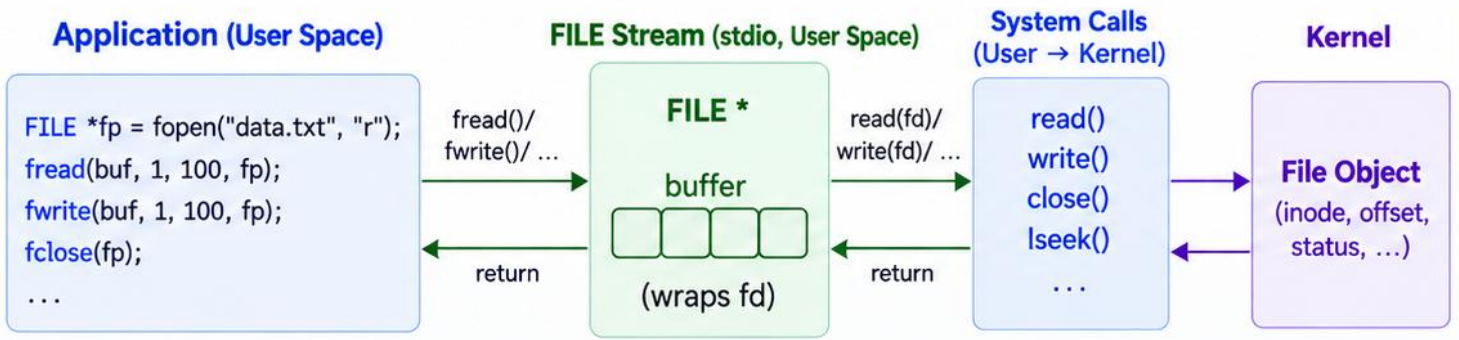
What is a FILE Stream?

A **FILE stream (FILE *)** is a user-space object that wraps a **file descriptor (fd)**.

Simplified struct _IO_FILE (glibc)

```
struct _IO_FILE {
  int   _fileno;          // underlying fd
  char  *_IO_read_ptr;    // read pointer
  char  *_IO_read_end;    // end of read area
  char  *_IO_write_base;  // start of write area
  char  *_IO_write_ptr;   // write pointer
  char  *_IO_buf_base;    // buffer base
  char  *_IO_buf_end;     // buffer end
  ....
};
```

- FILE contains a buffer and bookkeeping information.
- All stdio operations (fread, fwrite, fprintf, ...) use this buffer first.
- Actual I/O (read/write system calls) are issued only when necessary (e.g., buffer full, flush, close).



Example: fopen() and fclose()

```
FILE *fp = fopen("data.txt", "r");
  → open("data.txt", O_RDONLY)
  → returns FILE *   (fp->_fileno = fd)

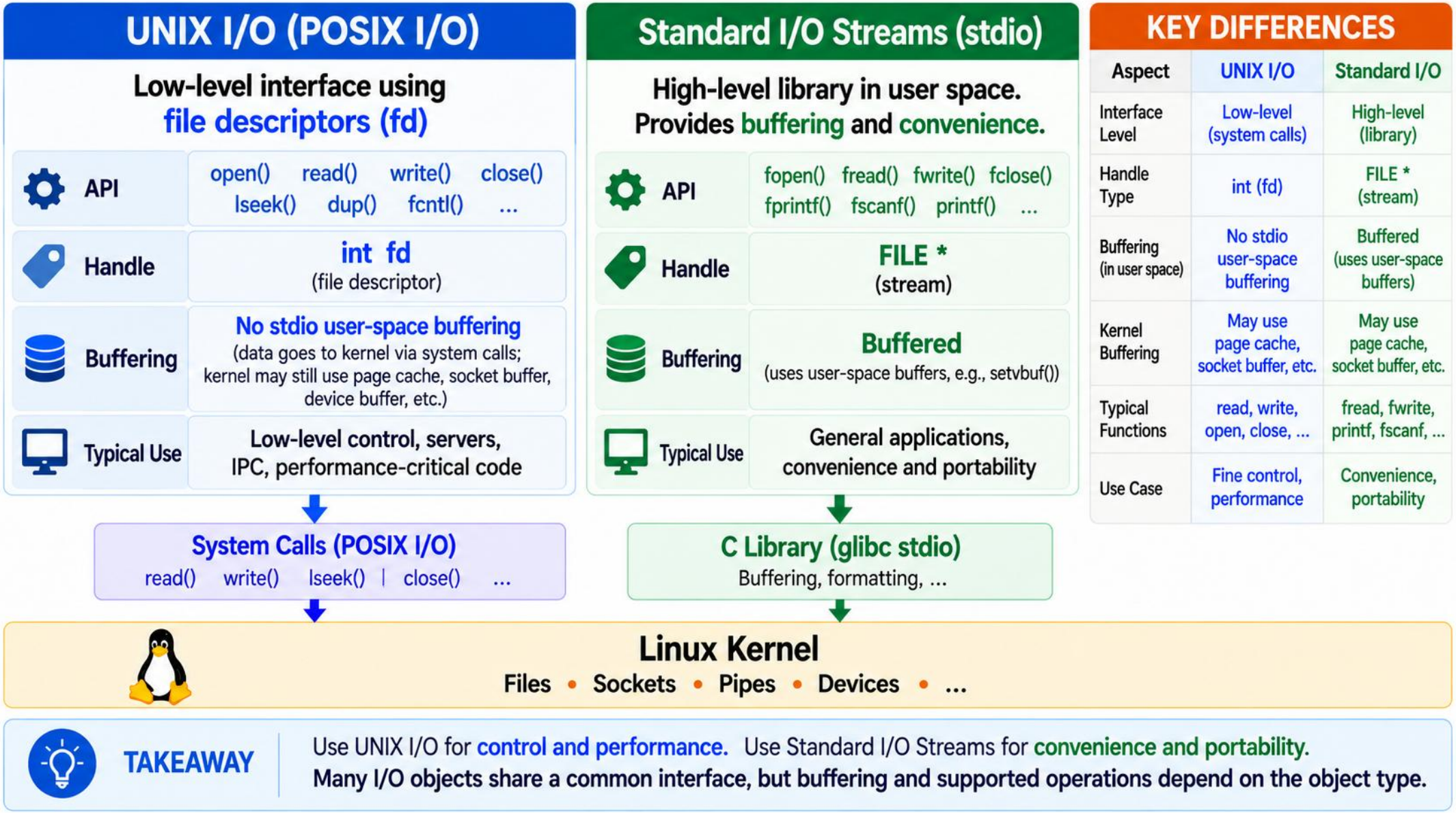
fclose(fp);
  → flushes buffer (if needed)
  → close(fd)
```

Default Streams

stdin → fd 0 stdout → fd 1 stderr → fd 2

A **FILE stream** provides **buffering and convenient functions**, while actual I/O is performed through the file descriptor by **system calls**.

Unix I/O and Standard I/O Streams



AI-generated illustration using ChatGPT (OpenAI)

One Interface, Many I/O Objects

A file descriptor (int) is a small integer that refers to an open I/O object. Different objects, same interface: read(), write(), close(), poll()/epoll(), ...

Regular File



fd = 3

open()
/data/log.txt

Data stored in filesystem (on disk)

Device (Character)



fd = 4

open()
/dev/input/event0

Device driver provides data (e.g., keyboard)

Socket (Network)



fd = 5

socket()
TCP 192.168.1.10:8080

Network communication (TCP/UDP, ...)

Event (Kernel Object)



fd = 6

eventfd()
eventfd object

Kernel event counter for notification between processes

Kernel (Common Abstraction)

File Descriptor Table (per process)

0	1	2	3	4	5	6	...

stdin stdout stderr

→ **struct file** (open file description)
- f_flags
- f_pos
- f_cred
- ..

→ **f_op (file_operations)**
- read()
- write()
- poll()
- mmap()
- release()
- ..

→ **Underlying Kernel Object**

inode / device / socket / eventfd / pipe /...



Examples in Code

```
int fd1 = open("data.txt", O_RDONLY);            // regular file
int fd2 = open("/dev/input/event0", O_RDONLY); // device
int fd3 = socket(AF_INET, SOCK_STREAM, 0);     // socket
int fd4 = eventfd(0, EFD_NONBLOCK);            // event
int ep  = epoll_create1(0);                    // epoll instance
```

Same Interface for All

```
ssize_t n = read(fd, buf, count);
ssize_t n = write(fd, buf, count);
int ready = poll(fds, nfds, timeout);
int ready = epoll_wait(ep, events, maxevents, timeout);
int ret = close(fd);
```

The same system calls work on all these FDs!

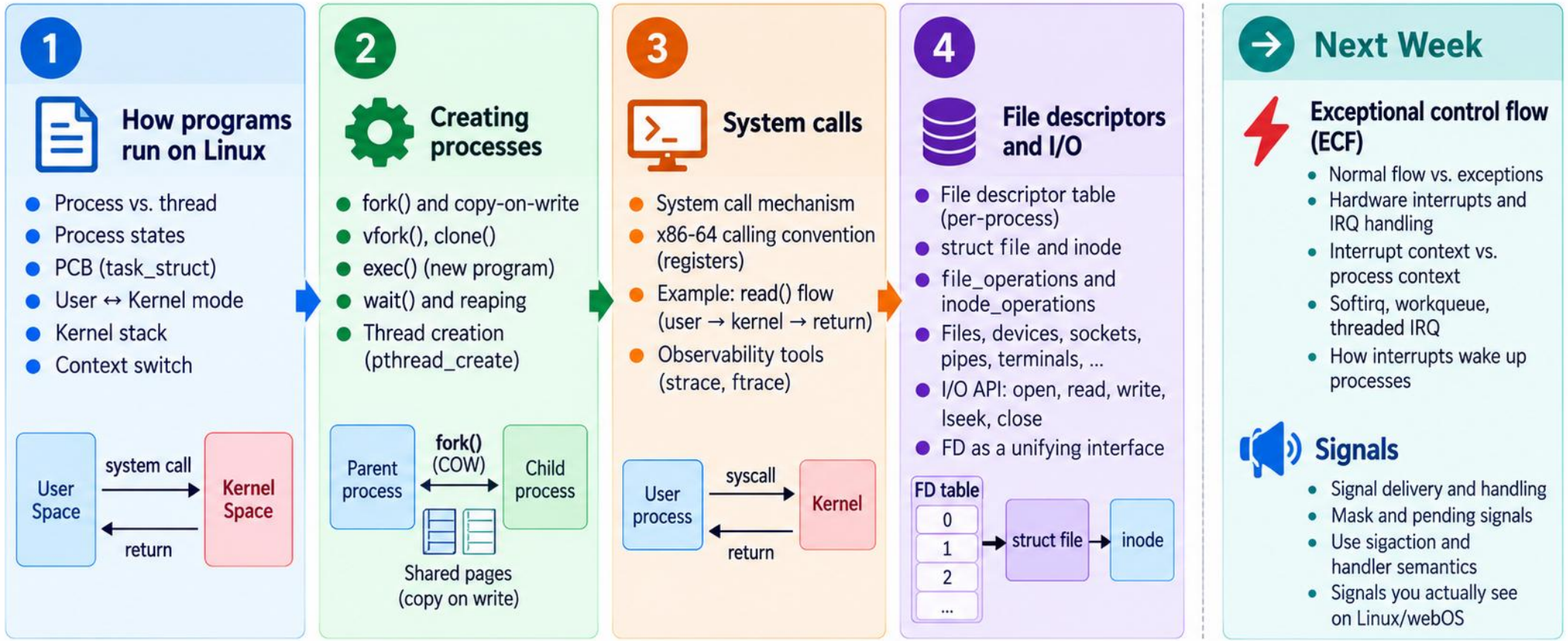
Key Takeaways

- A file descriptor is an integer handle.
- It refers to an open I/O object in the kernel.
- Different objects share the same interface.
- Enables efficient I/O multiplexing (select, poll, epoll, ...).

Default File Descriptors

0 → stdin 1 → stdout 2 → stderr

Week 1 Summary



- We learned how programs run on Linux: processes and threads, process creation and termination, system calls, and how files and I/O are accessed through file descriptors.
- These concepts provide the foundation for understanding system behavior and performance.
- Next week, we will explore what happens when the normal flow is interrupted: interrupts and signals, and how the kernel handles exceptional events.

End of Week 1

Sungyong Park, Ph.D. | parksy@sogang.ac.kr



